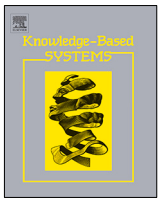
Knowledge-Based Systems 278 (2023) 110905

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys



Deep reinforcement learning for portfolio management

Shantian Yang *

School of Computing and Artificial Intelligence, Southwestern University of Finance and Economics, Chengdu, Sichuan, China
Kash Institute of Electronics and Information Industry, Kashgar, Xinjiang, China

ARTICLE INFO

Article history:
Received 19 May 2023
Received in revised form 10 August 2023
Accepted 12 August 2023
Available online 18 August 2023

Keywords:
Reinforcement learning
Graph representation learning
Portfolio management
Mutual information
Attention mechanism

ABSTRACT

Portfolio management facilitates trading off risks against returns for multiple financial assets. Reinforcement Learning (RL) is one of the most promising algorithms for portfolio management. However, these state-of-the-art RL algorithms only complete the task of portfolio management, i.e., acquire the different asset features of portfolio, without considering the global context information from portfolio, which leads to non-optimal portfolio representations; Moreover, the corresponding optimizations are implemented using only the loss function in the viewpoint of RL, without considering the relationships between the local asset information and global context embeddings, which leads to non-optimal portfolio policies. To deal with these issues, this paper proposes a Task-Context Mutual Actor–Critic (TC-MAC) algorithm for portfolio management. Specifically, TC-MAC algorithm is developed based on: (1) *representation learning* introduces a proposed Task-Context (TC) learning algorithm, which not only encodes the task (i.e., acquire different asset features) of portfolio, but also encodes the global dynamic context of portfolio, thus which helps to learn optimal portfolio embeddings; (2) *policy learning* introduces a proposed Mutual Actor–Critic (MAC) framework, which can measure the relationships between local embedding of each asset and global context embeddings by maximizing mutual information, the corresponding Mutual-Information loss function combines with RL loss function (i.e., Actor–Critic loss) to collectively optimize the whole algorithm, thus which helps to learn optimal portfolio policies. Experimental results on real-world datasets demonstrate the superior performance of TC-MAC algorithm over the well-known traditional portfolio methods and these state-of-the-art RL algorithms, at the same time, show its advantageous transferability.

1. Introduction

In financial market, the wealth is often dynamically allocated among diverse assets at the initial period and rebalanced afterwards, which focuses on making profits [1,2]. In general, these allocated assets are prone to market risks to some extent, which may lead to the high-risk and low-return situations. To this end, Portfolio Management (PM) is an effective method, because PM methods allocate wealth to diverse assets with different expected returns and levels of risks, and aim to achieve the low-volatility and long-term stable returns [3,4]. Early-stage PM methods solve different optimization problems to get effective portfolio weights for different assets [5,6]. But these traditional PM methods suffer from the issues of flexibility in both the goals achieving and time-sequential information encoding to some extent.

To alleviate these issues, PM methods adopt machine learning to achieve diverse goals, such as classification prediction for risks of different assets [7,8], and regression prediction for the expected returns of total wealth [9,10]. Especially for the interaction between the methods and the time-vary financial market, Reinforcement Learning (RL) is adopted by PM methods to perform portfolio policy, which can be regarded as a Markov Decision Process (MDP) [11,12]. Besides, in order to encode the time-sequential information, different machine learning techniques, such as Recurrent Neural Network (RNN) and Long Short-Term Memory (LSTM) [13], are employed to extract the features of time-vary trading data [9,14]. Furthermore, Deep Neural Networks (DNNs) are applied to RL, which improves its learning power [15,16]. Therefore, a lot of deep RL-based PM methods have achieved promising progress [17–19].

Deep RL-based PM methods are generally based on three types of RL algorithms: 1) *Value-based* algorithms, such as DQN [15], employ the parameterized neural networks to approximate the value function (i.e., accumulated wealth) [14,19]. While they may suffer from high bias due to non-stationary MDP transition for financial market. 2) *Policy-based* algorithms, such as REINFORCE [20], employ the parameterized neural networks to approximate the policy (i.e., allocated weights) [9]. Here the corresponding updating is based on the sampled episode returns,

* Correspondence to: School of Computing and Artificial Intelligence, Southwestern University of Finance and Economics, Chengdu, Sichuan, China.
E-mail addresses: yangst@swufe.edu.cn, yangshantian2009@gmail.com.

https://doi.org/10.1016/j.knosys.2023.110905
0950-7051/

while it could lead to high variance. 3) *Actor–Critic* algorithms, such as advantage actor–critic [21], employ different parameterized neural networks to respectively approximate the value function and policy [17,18]. Here the corresponding updating for both value function and policy can effectively trade off variance and bias. In general, these actor–critic algorithms first learn effective feature representation from the task (i.e., the features of different assets) of PM, and then produce optimal portfolio policies. In other words, the performance could be improved from the following two aspects:

On the one hand, these algorithms focus on capturing effective feature presentation from the different asset features of portfolio, which makes for learning well portfolio embeddings [12,17]. Specifically, the effective portfolio embeddings are learned by different techniques, such as Gate Recurrent Unit (GRU) [22] and attention mechanisms [23], from the features (e.g., open price and volume) of different assets in the portfolio. Unlike previous research [17,19], which did not consider the relationships between different assets, in this paper, the relationships will be first regarded as a dynamic portfolio network, and further defined as a temporal portfolio graph. Recently, different Graph Neural Networks (GNNs), such as Graph Convolution Network (GCN) [24], are derived from DNNs, which have been shown the state-of-the-art ability at learning graph-structured data [25,26]. So GNNs can suit to the dynamic graph of multi-asset portfolio context. This paper will design a GNN-based algorithm to capture the context representation from the defined temporal portfolio graph. In general, most existing DNN-based algorithms only employ the *local* feature representations of diverse assets as the final portfolio embeddings [14,18,19]. The ignored context of portfolio has the *global* features that can describe the whole relationships of diverse assets in the portfolio. Thus, it is naturally desirable that both global and local features from portfolio with different assets should be learned by certain algorithms (this progress is commonly deemed to as representation learning [27]), which is the one of research motivations proposed in this paper.

On the other hand, these algorithms focus on optimizing the corresponding objective functions for portfolio, which makes for deriving effective portfolio policies [9,28]. Specifically, based on the learned portfolio embeddings, the effective portfolio policies can be derived by minimizing the loss functions (i.e., objective functions), e.g., Mean Square Error (MSE) loss functions are for value-based algorithms [14], policy gradient loss functions are for policy-based algorithms [9], or combining with the above-mentioned two types loss functions are for actor–critic algorithms [17,18]. Due to the effective variance-bias tradeoff, most existing algorithms are optimized following the actor–critic paradigm [29,30], here the critic neural network first computes the global Q-value (i.e., total wealth) for multiple assets, then the corresponding partial gradient is used to judge the local portfolio policies (i.e., the allocated weight of each asset) produced by the actor neural network. That is to say, most existing RL algorithms for PM are only optimized by the actor–critic based loss (i.e., actor–critic loss function) from the standpoint of RL. However, these algorithms do not consider the representation relationships between the local assets' features and global context embeddings, which may lead to non-optimal portfolio policies. Moreover, how to calculate the relationships and design the corresponding loss function? Generally, the relationships could be calculated by maximizing mutual information [31] between local and global, but it introduces a new challenge to design the corresponding mutual-information loss function due to different local assets' features and global context embeddings.

Motivated by the above analysis, this paper proposes a Task-Context Mutual Actor–Critic (TC-MAC) algorithm for PM. TC-MAC algorithm is conducted based on both *representation learning* adopting a proposed Task-Context (TC) learning algorithm and *policy learning* adopting a proposed Mutual Actor–Critic (MAC) framework. Specifically, unlike most existing algorithms only to employ the features of different assets, the proposed TC learning algorithm not only encodes the task (i.e., the features of different assets) of PM as task embeddings using a designed attention-based Bi-GRU, but also encodes the global dynamic context of portfolio as context embeddings using a designed attention-based GCN, thus which helps to produce optimal portfolio embeddings. Moreover, unlike these algorithms to optimize only using the loss functions from the standpoint of RL, the proposed MAC framework could calculate the relationships between local assets' features and global context embeddings by maximizing mutual information, which lets the context embeddings involve more information about these diverse assets. The corresponding Mutual-Information (MI) loss function and Actor–Critic (AC) loss function collectively optimize the whole algorithm, thus which helps to produce optimal portfolio policies. This paper makes the following contributions:

(1) A TC learning algorithm is first proposed, which can learn not only the task (i.e., features of different assets) but also the context of portfolio, thus makes for producing optimal portfolio embeddings.
(2) And then a MAC framework is proposed, which can calculate the relationships between local assets' features and global context embeddings by maximizing mutual information. The corresponding MI loss function combines with AC loss function to collectively optimize the whole algorithm, thus makes for deriving optimal portfolio policies.
(3) Based on the above two proposed methods, this paper proposes a TC-MAC algorithm (i.e., the whole algorithm) for PM. Extensive experiments are implemented on different real-world datasets with multiple diverse assets. Results show that TC-MAC algorithm outperforms these state-of-the-art algorithms in terms of multiple metrics, and also illustrate that the trained TC-MAC algorithm can be effectively transferred to different test datasets.

The rest of this paper is organized as follows. We first introduce the problem formulation in Section 2. Then we propose a TC-MAC algorithm for PM in Section 3. After that we compare TC-MAC algorithm with other state-of-the-art algorithms in Section 4. Finally, we discuss the related work in Section 5, and draw some conclusions in Section 6.

2. Problem formulation

In this section, we first introduce a temporal portfolio graph for m assets in the portfolio, and then define the PM as an MDP.

2.1. Temporal portfolio graph

Unlike most previous research [11,12], which only consider the local representations of different assets in the portfolio, in this paper, in order to describe the global context representation of portfolio, we define the dynamic portfolio network at period t as a Temporal Portfolio Graph $\mathrm{TPG}^t = (\mathcal{V}, \mathcal{E}, \mathcal{B}, \mathcal{X}^t)$ shown in Fig. 1(left). Specifically, $\mathcal{V}$ is the set of m nodes, each node $i \in \mathcal{V}$ corresponds to the asset i; $\mathcal{X}^t = [x_1^t, \ldots, x_m^t]^T \in \mathbb{R}^{m \times d}$ is the corresponding node feature matrix at period t, which contains features that can characterize the dynamic features of different nodes (e.g., state space, action space and reward, which are defined in Section 2.2); $\mathcal{E}$ is the set of edges, each edge $e_{ij} \in \mathcal{E}$ is the similarity between node i and node j, the similarity e_{ij} is calculated by Eq. (9) defined in Section 3.1. Thus, the corresponding topology structure of graph TPG^t can be represented by a symmetric adjacency matrix $\mathcal{B} = \{e_{ij}\} \in \mathbb{R}^{m \times m}$.

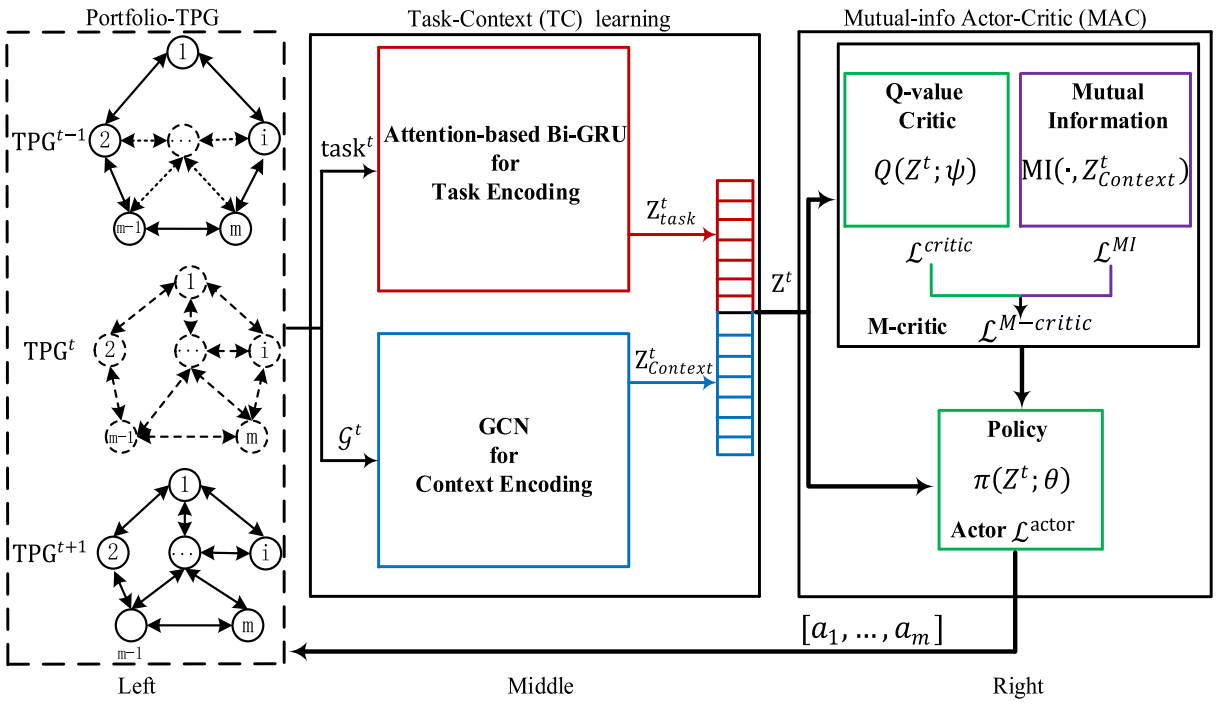


Fig. 1. Overview of TC-MAC algorithm for PM.

2.2. MDP for PM

Similar to previous research [14,19], PM can be defined as an MDP using a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \mathcal{H}, \rho_0, \gamma)$ for m assets in the portfolio, where $\mathcal{S} = \{\mathcal{S}_i\}_{i\in\{1,\dots,m\}}$ is a set of state spaces, $\mathcal{S}_i$ is the set of asset i's state, $\mathcal{A} = \{\mathcal{A}_i\}_{i\in\{1,\dots,m\}}$ is a set of action spaces, $\mathcal{A}_i$ is the set of asset i's actions (weights), $\mathcal{P}: \mathcal{S}\times\mathcal{A}\times\mathcal{S}\to\mathbb{R}$ is the transition probability distribution, $\mathcal{R} = \{\mathcal{R}_i\}_{i\in\{1,\dots,m\}}$ is a set of rewards, $\mathcal{R}_i$ is the set of asset i's rewards. $\mathcal{H}$ is the episode length, $\rho_0: \mathcal{S}\to\mathbb{R}$ is the distribution of the initial state $s^0 = \{s_1^0, \dots, s_m^0\} \in \mathcal{S}$ and $\gamma \in (0, 1]$ is the discount factor. Here state space, action space and reward function of PM for m assets in the portfolio are respectively defined as below:

• **State Space**: The state $s_i^t \in \mathcal{S}_i$ of each asset i includes previous open price o_i^t, closing price c_i^t, high price h_i^t, low price l_i^t, volume v_i^t, and some other financial indexes (e.g., Sharpe Ratio) at period t, i.e., $s_i^t = [o_i^t, c_i^t, h_i^t, l_i^t, v_i^t, SR]^T$. Thus, the state $s^t \in \mathcal{S}$ of m assets at period t is defined as:

$$s^t = [s_1^t, \dots, s_m^t]^T \tag{1}$$

• **Action Space**: The action $a_i^t \in \mathcal{A}_i$ of each asset i represents the weight of asset i. Thus, for m assets in the portfolio, the desired allocating weights $a^t = [a_1^t, \dots, a_m^t]^T \in \mathcal{A}$ is the allocating vector at period t, and $\sum_{i=1}^m a_i^t = 1$. Because of the price fluctuation at period t, the weights vector a^t at the beginning of period t will change into the reallocated weight $w^t = [w_1^t, \dots, w_m^t]^T$ at the end of that, and $\sum_{i=1}^m w_i^t = 1$, thus w^t can be defined as:

$$w^t = \frac{y^t \odot a^t}{y^t \cdot a^t} \tag{2}$$

where $y^t = [\frac{c_1^t}{c_1^{t-1}}, \dots, \frac{c_m^t}{c_m^{t-1}}]^T$ is the closing price fluctuating vector. $\odot$ is the Hadamard product, $\cdot$ is the dot product.

• **Reward Function**: The naive fluctuation of wealth is often defined as the reward function, which is to measure the return of actions [32]. In addition, unlike some related research [17,19] that did not consider transaction cost [33], which can measure the fraction of transaction amount in practice. In this paper, the fluctuation of wealth is defined as $y^t \cdot a^t$, transaction cost is defined as $\mu \sum_{i=1}^m |a_i^t - w_i^t|$, thus the total reward of m assets after period t is defined as:

$$r^t(s^t, a^t) = \sum_{i=1}^m r_i^t = y^t \cdot a^t - \mu \sum_{i=1}^m \left|a_i^t - w_i^t\right| \tag{3}$$

where $r_i^t(s_i^t, a_i^t) = a_i^t \times y_i^t - \mu\left|a_i^t - w_i^t\right|$ is the corresponding reward for each asset i at the end of period t. If r^t equals to 1, which means that there is no change for the wealth duration the period t; If $r^t > 1$, which means that the wealth is increasing duration the period t; If $0 < r^t < 1$, which means that the wealth is decreasing duration the period t.

Given a period $\mathcal{H}$, an agent allocates weights (i.e., portfolio policies $\pi(a|s)$) for m assets in the portfolio, and aims to maximize the accumulated return (i.e., wealth), which can be represented by the form of discount accumulated return, i.e., $G = \sum_{t=1}^{\mathcal{H}} \gamma^t r^t$. However, in the area of PM, due to the property that the wealth accumulated by period t would be reallocated at period t+1, thus the total wealth is represented as continued product form (i.e., $P_{\mathcal{H}} = P_0 \prod_{t=1}^{\mathcal{H}} r^t$) instead of summation, where P_0 is the initial wealth. To this end, we employ the logarithm of returns to convert continued product form into summation. Thus, the discount accumulated return G of m assets can be defined by $\ln(P_{\mathcal{H}}/P_0)$,

$$G = \ln \prod_{t=1}^{\mathcal{H}} r^t = \sum_{t=1}^{\mathcal{H}} lnr^t = \sum_{i=1}^m G_i \tag{4}$$

where $G_i = \ln\left(\prod_{t=1}^{\mathcal{H}} w_i^t \times r_i^t\right) = \sum_{t=1}^{\mathcal{H}} \ln(w_i^t \times r_i^t) = \sum_{t=1}^{252} \mathrm{r}_\mathrm{i}^{\mathrm{t}\times\mathrm{twz}}$ is the discount accumulated return for each asset i, which is the sum of logarithm of reward r_i^t discounted by w_i^t at each period t. $\mathrm{r}_\mathrm{i}^{\mathrm{t}\times\mathrm{twz}}$ is the accumulated reward calculated every twz trading days (twz is the trading window size set in Section 4.3). In finance, compound interest is used to calculate returns, that is to say, the current return is based on the previous day's return; Moreover, to avoid frequent trading that causes costs of trading to exceed benefits, thus the accumulated reward is based on every twz

(i.e., 100) trading days rather than each trading day. Therefore, we only calculate 252 times to get the accumulated reward of a whole episode.

Then, this paper is to design an algorithm, which first learns the final embedding Z^t of m assets in the portfolio, and then computes the critic Q-value $Q(Z^t;\psi)$ and the actor portfolio policy $\pi(Z^t;\theta)$, finally optimizes using both MI and AC loss functions. Mutual information is maximized to measure the relationships between local information $\langle s_i, a_i, r_i, s_i'\rangle$ and global embedding z_i, which makes global embedding involve more information about local information, so maximizing mutual information equals minimizing MI loss function for each asset i, thus,

$$\mathcal{L}_i^{MI} = -MI\left[\langle s_i, a_i, r_i, s_i'\rangle, z_i\right]. \tag{5}$$

For AC loss function, following [15], each asset i's Q-value $Q_i(\cdot)$ induced by a critic network approximates the discount accumulated return G_i; Each asset i's policy $\pi_i(\cdot)$ induced by an actor network approximates the weight. Thus, the loss function for m assets' critic Q-value is defined as:

$$\mathcal{L}^{\text{critic}} = \sum_{i=1}^{m}\mathbb{E}\left[r_i + \gamma \max_{a'} Q_i\left(s_i', a_i'; \overline{\psi}\right) - Q_i\left(s_i, a_i; \psi\right)\right]^2, \tag{6}$$

where $r_i + \gamma \max_{a'} Q_i\left(s_i', a_i'; \overline{\psi}\right)$ is the *target* Q-value induced by a target critic network, it is updated every C iterations. The loss function for each asset i's actor portfolio policy is updated by:

$$\nabla_\theta \mathcal{L}^{\text{actor}} = \mathbb{E}_{a\sim\pi}\left[\nabla_{\theta_i}\log\pi_i\left(a_i|s_i;\theta\right)Q_i\left(s_i, a_i;\psi\right)\right], \tag{7}$$

where θ is the parameter of each asset i's actor network.

3. A TC-MAC algorithm for PM

In the previous section, a graph TPG and RL elements for PM are defined in Eqs. (1)–(4). In this section, based on these definitions, this paper develops a TC-MAC algorithm for PM, as shown in Fig. 1, which is based on both *representation learning* using a proposed TC learning algorithm and *policy learning* using a proposed MAC framework. Specifically, TC learning algorithm not only encodes the task (i.e., the local features of m assets) of PM, but also encodes the global dynamic context of portfolio, thus which helps to produce optimal portfolio embeddings; MAC framework can measure the relationships between the local state of each asset and the global context embedding by maximizing mutual information, the corresponding MI and AC loss functions jointly optimize the whole algorithm, thus facilitates to derive optimal portfolio policies. In the following section, TC learning algorithm for portfolio representation is first described, and then MAC framework for portfolio policy is described.

3.1. A TC learning algorithm for portfolio representation

In this subsection, we introduce the proposed TC learning algorithm in detail. TC learning algorithm learns the portfolio embeddings via two parts: one is to encode the global context features of portfolio, the other is to encode the task (i.e., the local features of m assets) of PM. The former is implemented by a designed attention-based GCN, the latter is implemented by a designed attention-based Bi-GRU.

3.1.1. Context encoding using attention-based GCN

We focus on acquiring the context embeddings that can describe the global context features of portfolio with m assets. Based on the dynamic graph TPG defined in Section 2.1, a similarity graph $\mathcal{G}^t = (\mathcal{B}_m, \mathcal{X}^t)$ is constructed based on the defined node feature matrix $\mathcal{X}^t = [x_1^t, \ldots, x_m^t]^T \in \mathbb{R}^{m\times d}$, where B_m is the adjacency matrix of the similarity graph, $x_i^t = [s_i^{t-1}, a_i^{t-1}, r_i^{t-1}, s_i^t]$ is the feature of node i at period t. In order to align the nodes' features, each asset i's feature $[s_i^{t-1}, a_i^{t-1}, r_i^{t-1}, s_i^t] \in \mathbb{R}^{d_f\times 4}$ is first shifted into the corresponding embedding $e_i^t \in \mathbb{R}^{(d/2)\times 1}$:

$$e_i^t = \mathcal{W}_i\left[s_i^{t-1}, a_i^{t-1}, r_i^{t-1}, s_i^t\right]^T\mathcal{W}_i', \tag{8}$$

where $\mathcal{W}_i \in \mathbb{R}^{(d/2)\times 4}$ and $\mathcal{W}_i' \in \mathbb{R}^{d_f\times 1}$ are the learnable parameter matrices. In some certain periods, the similar assets often have the same trend [8], so how to measure the similarity between different assets? In general, a naive method is to adopt different similarity metrics, e.g., Euclidean distance and Cosine, to assess the shifted embeddings of assets [24,25]. While these conventional similarity metrics cannot work well enough due to certain latent features (e.g., high-order statistics) of assets ignored, which leads to uncompleted relevancy between different assets [34]. To this end, an alternative method, which is based on the kernel space, chooses the Heat Kernel [35] to derive the similarity matrix $M \in \mathbb{R}^{m\times m}$ among m nodes. Thus, the similarity e_{ij} between the asset i and asset j can be defined as:

$$e_{ij} = \exp\left(-\frac{\|e_i^t - e_j^t\|^2}{\lambda}\right) \tag{9}$$

where $e_{ij} \in [0, 1]$, the shifted embeddings e_i^t and e_j^t are feature vectors of assets i and j, λ is the time parameter. Expanding the exponential of similarity via Taylor series, it can be noted that the Heat kernel means an infinite dimension mapping, which helps to capture the high-order latent statistic features between different assets, at the same time, reduce the computation complexity to some extent.

In addition, if the similarity is too small, the relationship between different assets is low, which contributes less to the final global context representation. So, a threshold value τ is used to determine the similarity matrix M for m assets, that is to say, only when the similarity e_{ij} is larger than τ, it will be put into the M, where τ is set to 0.65. And the finally derived similarity matrix can be used as the adjacency matrix $\mathcal{B}_m$. Thus, based on the GCN [24], the lth layer output $Z_m^{(l)}$ can be defined as:

$$Z_m^{(l)} = Relu\left(\tilde{D}_m^{-\frac{1}{2}}\tilde{\mathcal{B}}_m\tilde{D}_m^{-\frac{1}{2}}Z_m^{(l-1)}W_m^{(l)}\right) \tag{10}$$

where $Relu$ is the activation function, $W_m^{(l)} \in \mathbb{R}^{(d/2)\times(d/2)}$ is the learned parameter matrix of the lth layer in GCN, the value of 0-th layer $(Z_m^{(0)})$ is initialized by $\mathcal{X}$, $\tilde{\mathcal{B}}_m = \mathcal{B}_m + I_m$, the corresponding degree matrix is $\tilde{D}_m$. In addition, the number of GCN's layer is set to 3, so the output embedding of GCN's 3-th layer is denoted as $Z_M = Z_m^{(3)} = \left[z_{1\_context}^t, z_{m\_context}^t\right]^T \in \mathbb{R}^{m\times(d/2)}$, $z_{i\_context}^t \in \mathbb{R}^{1\times(d/2)}$ is the embedding of asset i (i.e., the ith row of Z_M) at period t. For each node i's embedding $z_{i\_context}^t$, a non-linear transformation is adopted to get the corresponding attention value ω_i^t defined as:

$$\omega_i^t = q^T\cdot\tanh\left[W\cdot\left(z_{i\_context}^t\right)^T + b\right] \tag{11}$$

where $q \in \mathbb{R}^{h'\times 1}$ is a one shared attention vector, $W \in \mathbb{R}^{h'\times(d/2)}$ is the parameter matrix and $b \in \mathbb{R}^{h'\times 1}$ is the bias vector. The corresponding attention value ω_T^i is normalized by softmax function to acquire the weight α_i^t of each node i:

$$\alpha_i^t = softmax\left(\omega_i^t\right) = \frac{\exp(\omega_i^t)}{\sum_{j=1}^m\omega_j^t} \tag{12}$$

The α_i^t indicates the importance of embedding $z_{i\_context}^t$, for m nodes, the corresponding weight vector $\alpha^t = [\alpha_1^t, \ldots, \alpha_m^t]^T$ can be acquired by Eq. (12). In addition, based on GCN's output embedding Z_M at period t, the global context embedding for portfolio can be acquired by:

$$Z_{Context}^t = \sum_{i=1}^m\alpha_i^t\times z_{i\_context}^t = \alpha^t\cdot Z_M \tag{13}$$

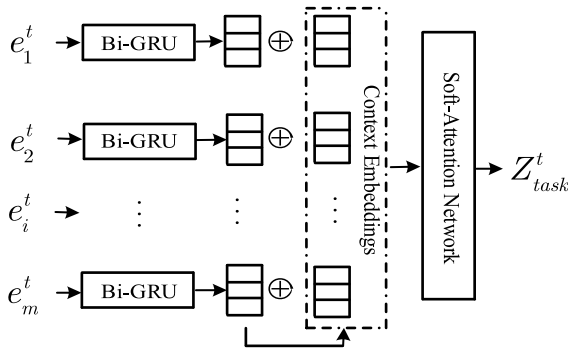


Fig. 2. Attention-based Bi-GRU for task embeddings.

Here $z^t_{i\_context}$ represents the local context embedding of asset i in the portfolio. In this way, we can learn the embedding which captures the global context information of portfolio.

3.1.2. Task encoding using attention-based bi-GRU

In addition to learning the global context presentation, we also focus on learning the task embedding that can describe the local features of m assets in the portfolio. Based on the $task^t = \{[s_i^{t-1}, a_i^{t-1}, r_i^{t-1}, s_i^t]\}_{i\in\{1,\ldots,m\}}$ with m asset states at period t, each asset i's feature $[s_i^{t-1}, a_i^{t-1}, r_i^{t-1}, s_i^t] \in \mathbb{R}^{d_f\times 4}$ is first shifted into the corresponding embedding $e_i^t \in \mathbb{R}^{(d/2)\times 1}$ via Eq. (8). Unlike previous research [18,19], the shifted embeddings are directly used for the next step, which may ignore some "deep" features. To alleviate this, the Bi-GRU [22] is adopted to learn each asset i's embedding $h_i^t \in \mathbb{R}^{(d/2)\times 1}$:

$$h_i^t = \mathcal{BG}_{j\in\mathcal{N}_i}\{e_j^t\} = \frac{\sum_{j\in\mathcal{N}_i}\left[\overrightarrow{GRU}\left\{e_j^t\right\} \| \overleftarrow{GRU}\left\{e_j^t\right\}\right]}{|\mathcal{N}_i|} \tag{14}$$

where $\mathcal{N}_i$ is the set of node i's neighbors from the similarity matrix $\mathcal{M}$ learned in Section 3.1.1, only when the similarity between nodes j and i is more than 0.65, the node j will be put into the set $\mathcal{N}_i$. In addition, the formulas of GRU are defined as:

$$\begin{aligned} m_j &= \sigma\left(\mathcal{U}_m e_j^t + \mathcal{W}_m u_{n-1} + b_m\right) \quad l_j = \sigma\left(\mathcal{U}_l e_j^t + \mathcal{W}_l u_{j-1} + b_l\right) \\ \hat{u}_j &= \tanh\left(\mathcal{U}_c e_j^t + m_j \odot \left(\mathcal{W}_c u_{j-1}\right) + b_c\right) \quad u_j = l_j \odot u_{j-1} + \left(1 - l_j\right) \odot \hat{u}_j, \end{aligned} \tag{15}$$

where m_j is the reset gate vector, l_j is the update gate vector, $\hat{u}_j$ is the new memory cell, $u_j \in \mathbb{R}^{(d/2)\times 1}$ is the output hidden state, $\odot$ is the Hadamard product, $\mathcal{W}_x \in \mathbb{R}^{(d/2)\times(d/2)}$, $\mathcal{U}_x \in \mathbb{R}^{(d/2)\times d}$ and $b_x \in \mathbb{R}^{(d/2)\times 1}$ $(x \in \{m, l, c\})$ are respectively the learnable parameters.

In general, the contributions of different assets to the discount accumulated return are not equivalent to each other. Intuitively, the more reward from certain assets in the tasks, which is more influence of agent's current portfolio policy (i.e., the weights of assets), that is to say, the weight of the corresponding asset is larger than that of other assets. In addition, it helps to filter noise in the tasks and capture the primary intentions of trading agent. Specifically, as described in Fig. 2, the shifted embeddings of assets in the task at period t are fed into the Bi-GRUs, then the embeddings (i.e., $H^t = [h_1^t, \ldots, h_m^t]^T$) of m assets are acquired, in addition, local context embeddings $Z_M = [z^t_{1\_context}, z^t_{m\_context}]^T$ for m assets have been learned in Section 3.1.1. For each asset i, the local context embedding $z^t_{i\_context}$ acquired by Eq. (10) is generated by using the feature information of *all assets* in portfolio; While the feature embedding h_i^t defined in Eq. (14) is generated by using the feature information of *these assets with similarity greater than 0.65*. Each asset has the feature information including state space, action (i.e., weight) space and reward defined in Section 2.2. In addition, based on the above analysis, the differences of "local context embedding" and "feature embedding" are as follows: the local context embeddings of all assets are aggregated to represent the global context feature of portfolio; While the feature embeddings of other similar assets are used to enhance the feature embeddings of current asset.

Thus, at period t, each asset i has both the local context embedding $z^t_{i\_context} \in \mathbb{R}^{(d/2)\times 1}$ and the feature embedding $h_i^t \in \mathbb{R}^{(d/2)\times 1}$, which are merged via concatenation and non-linear transformation:

$$c_i = \tanh\left(W_1\left[h_i^t \| z^t_{i\_context}\right] + b_1\right), \tag{16}$$

where $c_i \in \mathbb{R}^{d\times 1}$ is the merged embedding, which depends on not only the information from asset i in the task, but also the context information from asset i in the portfolio, $W_1 \in \mathbb{R}^{d\times d}$ and $b_1 \in \mathbb{R}^{d\times 1}$ are the learnable parameters. In addition, the asset information $s' \in \mathbb{R}^{(d/2)\times 1}$ can be acquired by averaging embeddings h_i^t of m assets:

$$s' = \frac{1}{m}\sum_{i=1}^{m} h_i^t. \tag{17}$$

Based on the c_i and s' acquired above, the corresponding weight β_i for each asset i can be learned via a soft-attention mechanism:

$$\beta_i = q_2^T \sigma\left(W_2 c_i + W_3 s' + b_2\right), \tag{18}$$

where $W_2 \in \mathbb{R}^{d\times d}$, $W_3 \in \mathbb{R}^{d\times(d/2)}$ and q_2, $b_2 \in \mathbb{R}^{d\times 1}$ are the learnable parameters. For all m assets, we can acquire the weight vector $\beta^t = [\beta_1^t, \ldots, \beta_m^t]^T$, and then linearly combine with the m assets' embeddings $H^t = [h_1^t, \ldots, h_m^t]^T$, the task embedding $Z^t_{task} \in \mathbb{R}^{(d/2)\times 1}$ of PM at period t is defined as:

$$Z^t_{task} = \mathcal{AG}_{\left\{\left\langle o_i^{t-1}, a_i^{t-1}, r_i^{t-1}, o_i^t\right\rangle\right\}_{i\in\{1,\ldots,m\}}}\left\{h_i^t\right\} = \sum_{i=1}^{m}\beta_i h_i^t, \tag{19}$$

Unlike most existing research [14,17], which learn the embeddings only from the information of assets themself, while in this paper, the learned task embeddings, which are based on both the information of assets and corresponding context information, could more precisely indicate the priority of different assets.

Finally, combining the task embedding Z^t_{task} defined in Eq. (19) and the context embedding $Z^t_{Context}$ defined in Eq. (13), the final portfolio embedding $Z^t \in \mathbb{R}^{d\times 1}$ at period t can be defined as:

$$Z^t = Contact(Z^t_{Context}, Z^t_{task}). \tag{20}$$

In summary, it can be noted that TC learning algorithm not only captures the task representation Z^t_{task} of PM, but also describes the global context representation $Z^t_{Context}$ of portfolio. Specifically, Z^t_{task} consists of both the features of all assets and priority features of all assets at each period t; $Z^t_{Context}$ consists of the whole context features of all assets in the portfolio. The learned final portfolio embedding Z^t helps to produce optimal portfolio policies by fusing different level information.

3.2. A MAC framework for portfolio policy

In MAC framework, the final portfolio embedding Z^t is respectively fed into the critic network to produce Q-value $Q^\pi(s^t, a^t; \psi) \approx Q^\pi(Z^t; \psi)$, and the actor network to produce portfolio policy $\pi(a|s; \theta) \approx \pi(Z^t; \theta)$. In order to acquire the maximum Q-value (i.e., wealth) and the optimal portfolio policies (i.e., weights of assets) in the PM, the previous research [17,18] employ the optimization only from the perspective of RL paradigm, which

may lead to non-optimal portfolio policies due to some relationships between the different feature representations ignored. To improve this, a MI loss function is proposed, which can describe the relationships between the local asset embeddings and global context embeddings by maximizing mutual information. Thus, the proposed MI loss function combines with RL loss function (i.e., AC loss function adopted in this paper) to optimize the whole algorithm.

3.2.1. MI loss

In general, mutual information can effectively measure the relationships between the local and global representations [31]. Following [36], mutual information between random variables U and V has the lower bound defined as:

$$\mathrm{MI}\left(U;V\right) \geq \mathbb{E}_{\mathbb{P}_{UV}}\left[\mathcal{T}_\phi\left(u,v\right)\right] - \log\left\{\mathbb{E}_{\mathbb{P}_{U\otimes\mathbb{P}_V}}\left[e^{\mathcal{T}_\phi(u,v)}\right]\right\}, \tag{21}$$

where $\mathbb{P}_{U\otimes}\mathbb{P}_V$ is the product of marginal distributions, $\mathbb{P}_{UV}$ is the joint distribution, $\mathcal{T}_\phi$ is a discriminator based on the DNN with parameter ϕ. The discriminator $\mathcal{T}_\phi$ is trained to distinguish the negative sample set from positive sample set, which can maximize mutual information. Similar to previous research [37], a bilinear layer could be as $\mathcal{T}_\phi$:

$$\mathcal{T}_\phi\left(u,v\right) = \sigma\left(u^{\mathcal{T}} W_{\mathcal{T}} v\right), \tag{22}$$

where σ is the sigmoid activation function, $W_{\mathcal{T}}$ is the learnable parameter.

The m asset embedding $\mathrm{H_t} = [\mathrm{h_1^t}, \ldots, \mathrm{h_m^t}]$ of PM at period t is learned by Eq. (14), and the context embedding $Z^t_{Context}$ of portfolio is learned by Eq. (13). Positive sample set is $Pos = \{[\mathrm{h_i^t}, Z^t_{Context}]\}_{i=1}^m$, $\mathrm{h_i^t}$ is learned by Eq. (14) based on set $\mathcal{N}_i$, negative sample set is $Neg = \{[\tilde{\mathrm{h}}_\mathrm{i}^\mathrm{t}, Z^t_{Context}]\}_{i=1}^m$, $\tilde{\mathrm{h}}_\mathrm{i}^\mathrm{t}$ is also learned by Eq. (14) for each asset i:

$$\tilde{h}_i^t = \mathcal{BG}_{j\in\overline{\mathcal{N}}_i}\left\{e_j^t\right\} = \frac{\sum_{j\in\overline{\mathcal{N}}_i}\left[\overrightarrow{GRU}\left\{e_j^t\right\} \| \overleftarrow{GRU}\left\{e_j^t\right\}\right]}{\left|\overline{\mathcal{N}}_i\right|}, \tag{23}$$

where $\overline{\mathcal{N}}_i$ equals the set $(task^t - \mathcal{N}_i)$, which is complement of set $\mathcal{N}_i$ defined in Section 3.1.2; $\tilde{\mathrm{h}}_\mathrm{i}^\mathrm{t} \in \tilde{H}^t = [\tilde{h}_1^t, \ldots, \tilde{h}_m^t]^T$. Thus, in the PM, the corresponding discriminator $\mathcal{T}_\phi$ can be determined as:

$$\mathcal{T}_\phi\left(\mathrm{h_i^t}, Z^t_{Context}\right) = \sigma\left[\left(\mathrm{h_i^t}\right)^T W_{\mathcal{T}} Z^t_{Context}\right]. \tag{24}$$

Based on the determined discriminator $\mathcal{T}_\phi$ and positive/negative sets, following the research [37], which maximizes the mutual information using the binary cross-entropy loss function. Therefore, we can maximize the mutual information between each asset embedding and the global context embedding, the corresponding MI loss function for m assets could be determined as:

$$\mathcal{L}^{MI} = \frac{1}{2m}\left\{\sum_{i=1}^m \mathbb{E}_{Pos}\left[\log \mathcal{T}_\phi\left(\mathrm{h_i^t}, Z^t_{Context}\right)\right] + \sum_{i=1}^m \mathbb{E}_{Neg}\left[\log\left(1 - \mathcal{T}_\phi\left(\tilde{\mathrm{h}}_\mathrm{i}^\mathrm{t}, Z^t_{Context}\right)\right)\right]\right\}. \tag{25}$$

In summary, the discriminator focuses on maximizing mutual information between the global context representation and local asset representation, which can stimulate the context encoder to capture the features presenting in all globally relevant portfolio. In addition, the context embeddings and asset embeddings would jointly present the feature information of PM from both local and global perspectives.

3.2.2. AC loss

For the RL-based optimization, unlike most existing research based on the Q-learning or DQN, which learn through temporal-difference learning by minimizing MSE [32]. To trade off the variance and bias, following the research [21], the optimization is based on the advantage Actor–Critic. Here, the Critic Q-value $Q^\pi(s^t, a^t; \psi) \approx Q^\pi(Z^t; \psi)$ is maximized to judge whether the corresponding Actor portfolio policy $\pi\left(a|s;\theta\right) \approx \pi(Z^t;\theta)$ is optimal. Therefore, these correspond to two objective loss functions respectively.

For the Critic loss, because all m assets of PM share the parameter ψ of critic network, there is some difference from the Eq. (6). Instead of the embedding $\mathrm{z_i^t}$ of each asset i, the final portfolio embedding Z^t is adopted to compute the Q-value via the critic network parameterized by ψ, thus corresponding Critic loss function is defined as follows:

$$\begin{aligned}\mathcal{L}^{critic} &= \mathbb{E}_{\mathcal{G}\cup Task}\{\left[Q\left(Z^t;\psi\right) - y_t\right]^2\} \\ &= \mathbb{E}_{(s,a,r,s')\sim D}\left\{\left[Q\left(s,a;\psi\right) - y_t\right]^2\right\},\end{aligned} \tag{26}$$

$$y_t = r_t + \gamma \mathbb{E}_{a'\sim\pi'(s';\theta')}\left[Q'\left(s',a';\overline{\psi}\right) - \delta\log\pi'\left(a'|s';\overline{\theta}\right)\right], \tag{27}$$

where $\mathcal{G}$ and $Task$ are respectively the similarity graph and the state for m assets at period t, which corresponds to the experience replay buffer D, a is the weight vector of m assets produced by the portfolio behavior policies π, s is the state of m assets, Q' is calculated by the *target* critic networks parameterized by $\overline{\psi}$, δ is the coefficient, which aims to trade off expected return and entropy [29].

Based on the optimization for representation relationships, the corresponding MI loss function defined in Eq. (25) will be integrated into the Critic loss function defined in Eq. (26), the final critic loss function is determined as follows:

$$\mathcal{L}^{M-critic} = \mathcal{L}^{critic} + \lambda_m \mathcal{L}^{MI} + \lambda_\psi \|\psi\|^2, \tag{28}$$

where λ_m is the coefficient of MI loss terms, λ_ψ is the penalty coefficient of L2 regularization term about all the learnable parameters.

For the Actor loss, similar to the Critic loss, there is also some difference from the Eq. (7), for all m assets, the corresponding weight vector a is the portfolio policy $\pi\left(Z^t;\theta\right)$ produced the actor network parameterized by the shared θ, that is to say, each item in the weight vector a is the weight of one asset, thus the actor portfolio policies are updated by using the following gradient:

$$\begin{aligned}\nabla_\theta \mathcal{L}^{\text{actor}} &= \mathbb{E}_{\mathcal{G}\cup Task}\left\{\nabla_\theta \log\pi\left(Z^t;\theta\right)\left[A\left(Z^t\right) + \epsilon\log\pi\left(Z^t;\theta\right)\right]\right\} \\ &= \mathbb{E}_{\boldsymbol{a}\sim\pi(\boldsymbol{o};\theta)}\left\{\nabla_\theta \log\pi\left(a|s;\theta\right)\left[A\left(s,a\right) + \epsilon\log\pi\left(a|s;\theta\right)\right]\right\}\end{aligned} \tag{29}$$

$$A\left(s,a\right) = Q\left(s,a;\psi\right) - \sum_{i=1}^m \pi_i\left(a_i|s_i\right) Q_i(s_i,a_i), \tag{30}$$

where $A\left(s,a\right)$ is the advantage function to deal with the issue of credit allocation [38], $\sum_{i=1}^m \pi_i\left(a_i|s_i\right) Q_i(s_i,a_i)$ is a baseline, which is the linear weighted sum of accumulated return for m assets, $Q_i(s_i,a_i)$ is the accumulated return of each asset i, the corresponding weight coefficient $\pi_i\left(a_i|s_i\right)$ is the action derived by the portfolio policy. In addition, to alleviate the underlying overgeneralization [39], the actions are sampled from the portfolio policy at current period t instead of the previous experience buffer.

The whole TC-MAC algorithm is summarized by Algorithm 1. During the training process, the collected information (e.g., open/closing/high/low prices and volume defined in Section 2.2) is used to acquire the final portfolio embeddings via TC learning algorithm proposed in Section 3.1, the final portfolio embedding is used to compute Q-value and portfolio policies, and then the whole algorithm is optimized by MAC framework proposed in Section 3.2. Specifically, based on the graph TPG defined in Section 2.1, the global context embedding $Z^t_{Context}$ for portfolio is

Algorithm 1. The Training Process of TC-MAC Algorithm for PM

1: **Input**: Graph TPG=$(\mathcal{V},\mathcal{E},\mathcal{B},\mathcal{X}^t)$ #***defined in Section 2.1***.

2: **Output**: Parameter ψ of Critic network Q, Parameter θ of Actor network π.

3: Randomly initialize Critic network Q_ψ, Actor network π_θ, target networks $Q_{\overline{\psi}}$ and $\pi_{\overline{\theta}}$; Mutual discriminator $\mathcal{T}_\phi$, Replay buffer D

4: **for** episode=1 to M **do**

5: Initialize a random process $\mathcal{N}$ for action exploration

6: Receive initial state s^0

7: **for** t=1 to T **do**

8: Select action $a_i^t=\pi(s_i^t;\theta)+\mathcal{N}^t$ for each asset i # ***total m assets, the corresponding state s^t is defined in Eq. (1)***

9: Execute the reallocated weight w^t derived from action $a^t=[a_1^t,\ldots,a_m^t]$ # ***defined in Eq. (2)***

10: Receive reward r^t defined in Eq. (3), and next state s^{t+1}

11: Store transition (s^t, a^t, r^t, s^{t+1}) in reply buffer D

12: $s^t \leftarrow s^{t+1}$

#TC Learning Algorithm

13: Sample a random minibatch of B transition from D

14: Compute the global context embedding $Z_{Context}^t$ for portfolio using Eqs. (8)-(13)

15: Compute the task embedding Z_{task}^t for portfolio using Eqs. (14)-(19)

16: Compute the final portfolio embedding $Z^t=Contact(Z_{Context}^t,Z_{task}^t)$ using Eq. (20)

#MAC Framework

17: Compute the Q-value $Q^\pi(s^t,a^t;\psi)\approx Q^\pi(Z^t;\psi)$, and portfolio policy $\pi(a|s;\theta)\approx\pi(Z^t;\theta)$.

18: Update Critic network Q_ψ and Mutual discriminator $\mathcal{T}_\phi$ by minimizing Eq. (28). # ***this is derived from Eqs. (21)-(27)***

19: Update Actor network using the sampled policy gradient defined in Eq. (29)

20: Update target Actor and Critic networks by soft-update:

21: $\overline{\psi}\leftarrow\tau\psi+(1-\tau)\overline{\psi}$

22: $\overline{\theta}\leftarrow\tau\theta+(1-\tau)\overline{\theta}$

23: end for

24: end for

25: Return ψ and θ

learned in Section 3.1.1, at the same time, the task embedding Z_{task}^t is learned in Section 3.1.2, the final portfolio embedding Z^t is acquired by contacting $Z_{Context}^t$ and Z_{task}^t. The final portfolio embedding is used to produce the portfolio policies, after the portfolio policies (i.e., action defined in Section 2.2) are performed, the reward is computed by the reward function defined in Section 2.2, then the Q-value is computed. Finally, the corresponding MI loss function defined in Section 3.2.1 and AC loss function defined in Section 3.2.2 jointly optimize the whole algorithm.

Based on the above description of Algorithm 1, the time complexity and space complexity are discussed as follows: In TC learning algorithm, the global context embedding $Z_{Context}^t$ is acquired by the attention-based GCN defined in Eqs. (8)–(13), the corresponding time complexity is $\mathcal{O}(|\mathcal{E}|\,Ld^2+(md_f+mh')d+m+h')$, and the space complexity is $\mathcal{O}(Ld^2+(mL+h'+\mathrm{m})\mathrm{d}+(d_f+h')m+Lm^2)$, where $|\mathcal{E}|$ is the number of TPG's edges, L is the number of GCN layers, d is the dimensionality of final portfolio embedding, m is the number of assets, d_f is the dimensionality of learnable parameter matrix $\mathcal{W}_i'$ defined in Eq. (8), h' is the dimensionality of learnable parameter matrix W defined in Eq. (11); the task embedding Z_{task}^t is acquired by the attention-based Bi-GRU defined in Eqs. (14)–(20), the corresponding time complexity is $\mathcal{O}(m|\mathcal{N}_i|d^2+md+m)$, and the space complexity is $\mathcal{O}(md^2+md)$, where $|\mathcal{N}_i|$ is the number of neighbor nodes whose similarity to node i is greater than 0.65. In MAC framework, the algorithm is optimized by MI loss function defined in Eqs. (22)–(25), the corresponding time complexity is $\mathcal{O}(m|\overline{\mathcal{N}}_i|d^2+md)$, and the space complexity is $\mathcal{O}(md^2)$, where $|\overline{\mathcal{N}}_i|$ is the number of non-neighbor nodes for node i; at the same time, the whole algorithm is also optimized by AC loss function defined in Eqs. (26)–(30), the corresponding time complexity is $\mathcal{O}(d+md)$, and the space complexity is $\mathcal{O}(d+md)$. Therefore, taking into account the total number (i.e., M) of episodes and the number (i.e., T) of runs per episode, the final overall time complexity of TC-MAC algorithm is $\mathcal{O}(MT[\left(m\left|\overline{\mathcal{N}}_i\right|+m\left|\mathcal{N}_i\right|+|\mathcal{E}|\,L\right)d^2+\left(md_f+mh'+m\right)d+m+h'])$, and the final overall space complexity is $\mathcal{O}(MT[(m+L)d^2+(mL+m+h')d+(h'+d_f)m+Lm^2])$.

4. Experiments

The proposed TC-MAC algorithm is evaluated on multiple different datasets. Specifically, TC-MAC algorithm is first trained on a training dataset, and then the trained algorithm is tested on a homogeneous test dataset, which can be considered as homogeneous transfer; In addition, the trained algorithm is also tested on a heterogeneous test dataset, which can be considered as heterogeneous transfer. The above three datasets are defined in Section 4.1, the homogeneous test dataset has the same assets with the training dataset except for the time periods, while the heterogeneous test dataset has both totally different

Table 1
Summary of the underlying portfolios with 12 assets from various sectors.

Ticker	MSFT	AAP	NOV	AMAT	XOM	FORD	AMZN	JPM	KO	IYY	SHY	MMM
Type	Share	Share	Share	Share	Share	Share	Share	Share	Share	ETF	ETF	Share
Sector	IT	IT	Equipment	Material	Energy	Auto	Retail	Bank	Drink	Dow Jones	US bond	Multi
Company	Microsoft	Apple	National Oilwell Varco	Applied Material	Exxon Mobil	Ford Motor	Amazon	JPMorgan Chase	Coca-Cola	iShare Dow Jones	iShare Treasury Bond	3M

assets and time periods compared with the training dataset. But most previous research, such as [14,17,19], only do homogeneous transfer. While heterogeneous transfer can further measure the effectiveness and transferability of TC-MAC algorithm for PM with different types and amounts of assets.

4.1. Datasets

Following the previous research [8,9,11,28], which combine multiple diverse assets from S&P500 index as different datasets to assess the performances of algorithms. Thus, in this paper, the adapted datasets also contain similar assets, which come from various industrial sectors including Automotive, IT, Bank and so on. As shown in Table 1, i.e., Microsoft and Apple (from IT Sector), Amazon (from Retail sector), 3M (from Multiple-sources sector), National Oilwell Varco (from Equipment manufacturing), Applied Material (from Material sector), Exxon Mobil (from Energy sector), Ford Motor (from Automotive sector), JPMorgan Chase (from Bank sector), Coca-Cola (from Drink sector), and two Exchange Trade Funds (ETFs): IYY and SHY. These diverse assets could arguably describe the entire financial market, and then the portfolios with diverse cardinalities due to the major income derived from certain assets.

Therefore, both training and test datasets are designed based on the specific time periods and diverse assets, where these assets have the same frame length, and come from different sectors so that the final policies are somewhat sector-neutral. The three datasets are described as follow: 1) A *training dataset* contains five assets: MSFT, 3M, NOV, AMAT and IYY, the corresponding historical data is from 1/1/2009 to 31/12/2017, i.e., 9 years (total 2268 trading days). 2) Based on the same five assets with the training dataset, a *homogeneous test dataset* is designed, which includes historical data between 1/1/2018 to 31/12/2019, i.e., 2 years (total 504 trading days). 3) We also design a *heterogeneous test dataset*, which contains different assets: XOM, FORD, AMZN, AAP, JPM, KO and SHY, the corresponding historical data is from 1/1/2018 to 31/12/2019. These two different test datasets are to evaluate not only the effectiveness but also the transferability of these algorithms.

For each asset, the corresponding data is fetched from Yahoo Finance,[1] which contains trading volume and four daily prices, i.e., Open-High-Low-Close prices. In addition, each price series is represented a unique characteristic or pattern, which helps to improve the commonality of these datasets.

4.2. Compared algorithms and performance metrics

To evaluate the effectiveness and transferability of TC-MAC algorithm, we compare it with the state-of-the-art Deep RL (DRL) algorithms and some well-known traditional portfolio methods as follows.

Traditional Methods: (1) *Uniform Constant Rebalanced Portfolios* (UCRP), which is as a benchmark, reallocates a uniform portfolio with equal weights of different assets at the starting of each period [4]. 2) *Exponential Gradient Portfolios* (EGP) [3], which is a follow-the-winner method, reallocates the more weights to the assets that can make higher profit at the previous period. 3) *On-Line Moving Average Reversion* (OLMAR) [5], which is a follow-the-loser method, increases the relative weights of assets that make lower profit at the previous period.

DRL algorithms: (1) KD-PPO [17], which employs the knowledge distillation technique to improve the performance of Proximal Policy Optimization (PPO) [30], and then predicts the weights of multiple diverse assets. (2) Adaptive DDPG [18], which incorporates optimistic or pessimistic into Deep Deterministic Policy Gradient (DDPG) [40], and then predict the weights of multiple diverse assets. (3) 3DQN [19], which employs the classical dueling double DQN [38] to predict the weights of multiple diverse assets. 4) RRL [14], which employs the classical recurrent RL and particle swarm optimization to predict the weights of multiple diverse assets. 5) TD3 [41] is as the compared algorithm due to its relatively robust performance.

Metrics: To measure the performances of algorithms, we adopt five of the most universal metrics in the real financial market as follows. (1) *Net Profit* (NP), i.e., $W_T - W_0$, where W_T is the accumulated final wealth at the end of period T, W_0 is the initial wealth at the start of period 0. NP has been adopted by [4,19,28]. (2) *Percentage Yield* (PY), i.e., $(W_T/W_0)^{\frac{252}{T}}$-1, indicates the annualized percentage gain in the condition of 252 trading days in a year. PY has been adopted by [1,17,18]. (3) *Sharpe Ratio* (SR), i.e., $(PY - 0.001)/\mathrm{Std}(R_{1,\ldots,T})$, indicates the annualized risks adjusted returns, at the same time, the annualized risk-free return is set to 0.001. SR has been adopted by [7,11,14]. (4) *Calmar Ratio* (CR), i.e., PY/MDD, indicates the levels of risks taken to get a return. CR has been adopted by [9]. (5) *Maximum Drawdown* (MDD), i.e., $(PV - LV)/PV$, indicates the downside risks over a specific period, that is to say, the maximum loss is from peak value (PV) to the lowest value (LV) for the portfolio. MDD has been adopted by [3,9]. In general, the better the performance of algorithm is, the higher value of all these metrics are except for MDD due to the risk-averse nature of investors.

4.3. Implementation and hyperparameter setting

Following some related research [17,42,43], multiple different hyperparameters are fine-tuned to achieve the best results, and then the corresponding values can be determined. Specifically, the time parameter λ in Eq. (9) is set to 2, the dimensionality d of portfolio embedding is set to 256, the discount factor γ is fine-tuned and set to 0.90 (Note that it is different from others values (e.g., 0.99) due to the cost of trading and compound interest in finance), the updating iteration C for the target networks is set to 10000, the coefficient δ defined in Eq. (27) is set to 0.85, λ_m and λ_ψ defined in Eq. (28) are respectively set to 0.70 and 0.001, the coefficient ϵ defined in Eq. (29) is set to 0.50, the total training steps T is 252000, the initial wealth W_0 is set to US$10000.00, the factor μ of transaction cost is set to 0.003, the batch size is set to 128, and the trading window size (i.e., twz) is set to 100. Following similar research [18,19], for a fair comparison, we make the shared hyperparameters same for other four DRL algorithms and traditional methods. In addition, other unique hyperparameters of these compared algorithms are also fine-tuned to acquire the best results.

[1] accessible from https://finance.yahoo.com/

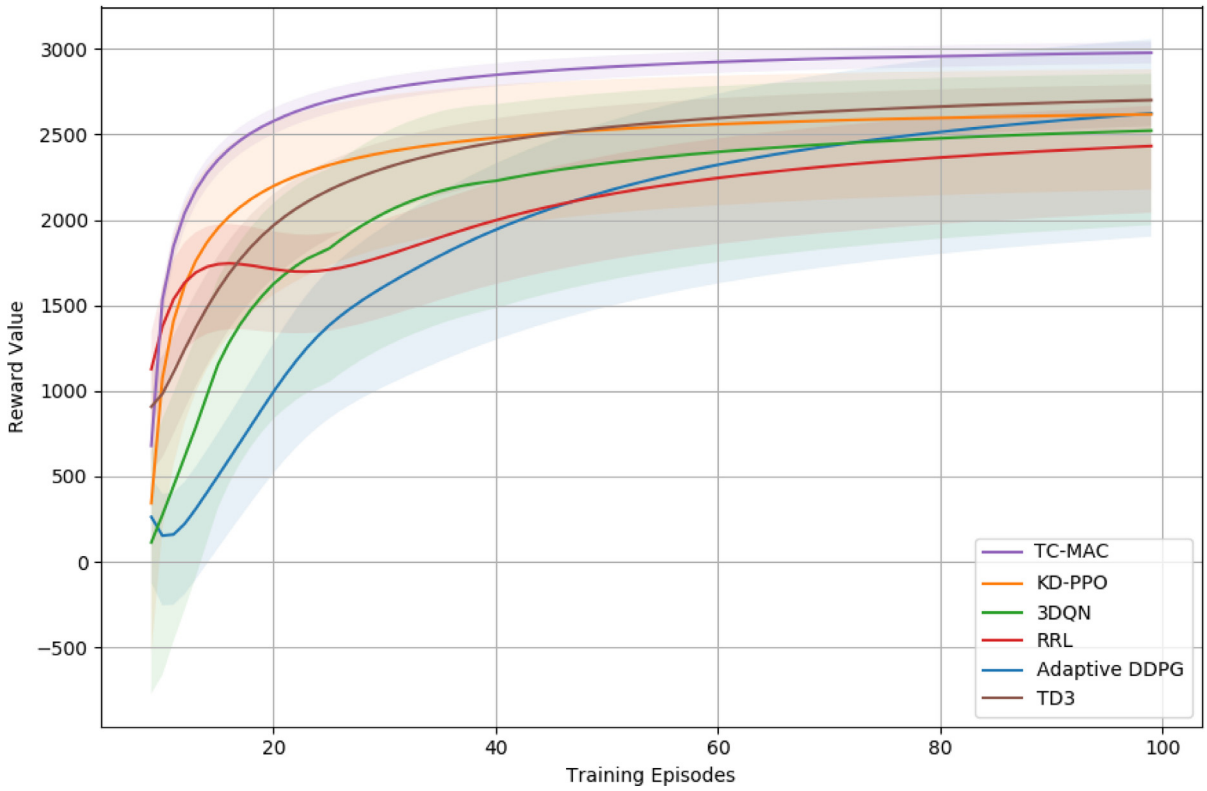


Fig. 3. Training results.

About the implementation, similar to previous research [9,14], we first train the proposed TC-MAC algorithm on the training dataset defined Section 4.1. The total training episodes are 100, each episode is 2520, that is to say, there is total 252000 training steps. These compared DRL algorithms are also trained in the same experimental setting.

And then we evaluate these trained algorithms and traditional methods on the homogeneous test dataset defined in Section 4.1. At the same time, we also evaluate these trained algorithms on the heterogeneous test dataset defined in Section 4.1. The former can be considered as *homogeneous transfer*, where both training and homogeneous test datasets are with the multiple same assets but different time periods; While the latter can be considered as *heterogeneous transfer*, where both training and heterogeneous test datasets are with both totally different assets and time periods. These homogeneous and heterogeneous transfer aim to measure the performances of these algorithms. In addition, the test process is repeated five times, and then the final results are achieved by aggregating the above results.

4.4. Training results

To describe the training results of these compared DRL algorithms on the training dataset defined Section 4.1, based on the reward r^t defined in Eq. (3) at timestep t, *average episode reward* is to sum all timestep reward r^t in an episode. Thus, as shown in Fig. 3, these average episode rewards for different algorithms are plotted via solid lines of different colors, the corresponding standard deviations are indicated by the shades. It can be noted that the curves of other compared DRL algorithms are below that of TC-MAC algorithm, which suggests that TC-MAC algorithm is better than other compared algorithms.

Specifically, it can be noted that KD-PPO is better than other five compared algorithm before 43 training episodes, while at the later training stage, TD3 achieves the best results in the five compared algorithm. After reaching the stable state, the reward of KD-PPO is about 2629.58, and standard deviation is about 197.35, that of TD3 is about 2710 $\pm$ 298.50. Although Adaptive DDPG has the close stable reward about 2632.74 $\pm$ 203.50, it suffers from poor convergence compared with KD-PPO. The explanation is that the adopted knowledge distillation technique (KD-PPO) work better than the adaptive ability (Adaptive DDPG) in the training process. After achieving the stable state, TC-MAC is better than both KD-PPO and TD3. 3DQN and RRL have the stable rewards about 2507.94 $\pm$ 409.76 and 2451.13 $\pm$ 401.14, respectively. It can be clearly seen that TC-MAC has the stable reward about 2997 $\pm$ 83.64, which suggests that it has the lowest standard deviation and the largest reward. In addition, it also can be noted that RRL is better than Adaptive DDPG before the 43th episode, while the latter is better than the former at the whole following episodes; Moreover, Adaptive DDPG is better than 3DQN after training 76 episodes, and finally the curve of Adaptive DDPG is approximate to the curve of KD-PPO. The explanation is that the adopted particle swarm optimization (RRL) can improve the speed of convergence compared with 3DQN and Adaptive DDPG at the early training episodes.

On the whole, it can be seen from Fig. 3 that all the curves of these algorithms are increasing at the early training-step episodes, and then reaching the different maximum values, finally keeping stable. For TC-MAC and KD-PPO, it can be noted that the curve of TC-MAC is rising sharply, i.e., the reward reaches to about 2598.12 after training 20 episodes; The similar situation also happens to both KD-PPO and 3DQN, the corresponding rewards reach to about 2205.98 and 1594.65 after training 20 episodes, respectively. Although RRL and Adaptive DDPG have some downward trend at the early training episodes (i.e., episodes from 6 to 10 for Adaptive DDPG, and episodes from 17 to 22 for RRL), the whole trends are upward before 20 episodes. And when episodes between 20 and 40, it also can be noted that all curves of these

Table 2
Results for heterogeneous transfer.

Metrics	Heterogeneous transfer									
	TC-MAC	Adaptive DDPG	KD-PPO	3DQN	RRL	UCRP	Winner-EGP	Loser-OLMAR	TD3	Changed
NP	**3207.32**	1878.65	1648.61	−727.34	1321.52	564	−3152.94	−4975.61	2103.50	↗52.5%
PY	**0.149**	0.090	0.079	−0.037	0.064	0.028	−0.173	−0.291	0.101	↗47.5%
SR	**1.518**	0.809	0.921	−0.319	0.606	0.212	−1.773	−2.339	0.909	↗64.8%
CR	**0.756**	0.435	0.560	−0.227	0.282	0.097	−0.348	−0.547	0.498	↗35.0%
MDD	**0.197**	0.207	0.141	0.163	0.227	0.289	0.497	0.532	0.203	↘39.7%

Table 3
Results for homogeneous transfer.

Metrics	Homogeneous transfer									
	TC-MAC	Adaptive DDPG	KD-PPO	3DQN	RRL	UCRP	Winner-EGP	Loser-OLMAR	TD3	Changed
NP	**5168.83**	3987.53	4389.61	3025.64	2487.18	2050.47	2475.37	1142.86	4009.70	↗17.8%
PY	**0.232**	0.183	0.200	0.141	0.117	0.098	0.117	0.056	0.184	↗16.0%
SR	**2.578**	1.664	2.105	1.343	1.017	0.951	1.026	0.523	1.794	↗22.5%
CR	**1.059**	0.638	0.641	0.576	0.394	0.354	0.397	0.166	0.780	↗35.8%
MDD	**0.219**	0.287	0.312	0.245	0.297	0.277	0.295	0.337	0.236	↗7.2%

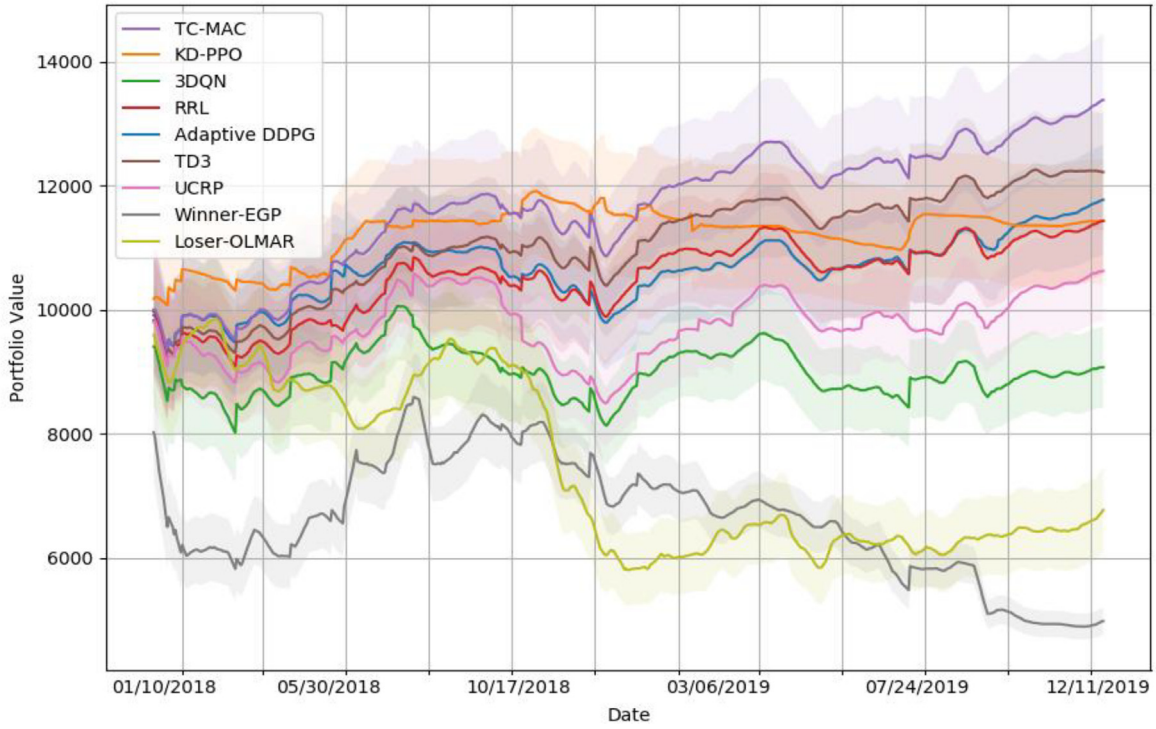


Fig. 4. Portfolio values in heterogeneous transfer.

algorithms rise slowly compared with those at the early training episodes (i.e., before 20 episodes); At the following training episodes, it can be seen that all algorithms can keep the reward stable. The possible explanation is that the experimental data is insufficient and lack diversity at the early training, which reduces the optimization ability and feature representation for financial market. Furthermore, it also can be seen from Fig. 3 that the effect of training gradually improves with the abundant accumulated data at the later training episodes, these data also provide diverse features for training. So based the above analysis about the trends of different curves, which suggests that all algorithms achieve the convergence to some extent. While the proposed TC-MAC algorithm outperforms than other compared algorithms, some partial reason is that the latter ignore the global dynamic context of portfolio, which leads to the learned portfolio embeddings that are effective to some extent.

4.5. Test results

After the training completed, these trained algorithms and traditional methods are respectively evaluated on the homogeneous test dataset and heterogeneous test dataset, which corresponds to the homogeneous transfer (i.e., the training and evaluation use *the same multiple assets*) and heterogeneous transfer (i.e., the training and evaluation use *totally different multiple assets* including the number of assets). The heterogeneous transfer results are shown in Table 2 and Figs. 4–6, the homogeneous transfer results are shown in Table 3 and Figs. 7–9.

4.5.1. Heterogeneous transfer

Table 2 reports five performance metrics of TC-MAC algorithm and other compared algorithms during the heterogeneous transfer. The observed values are spatially summed at each timestep

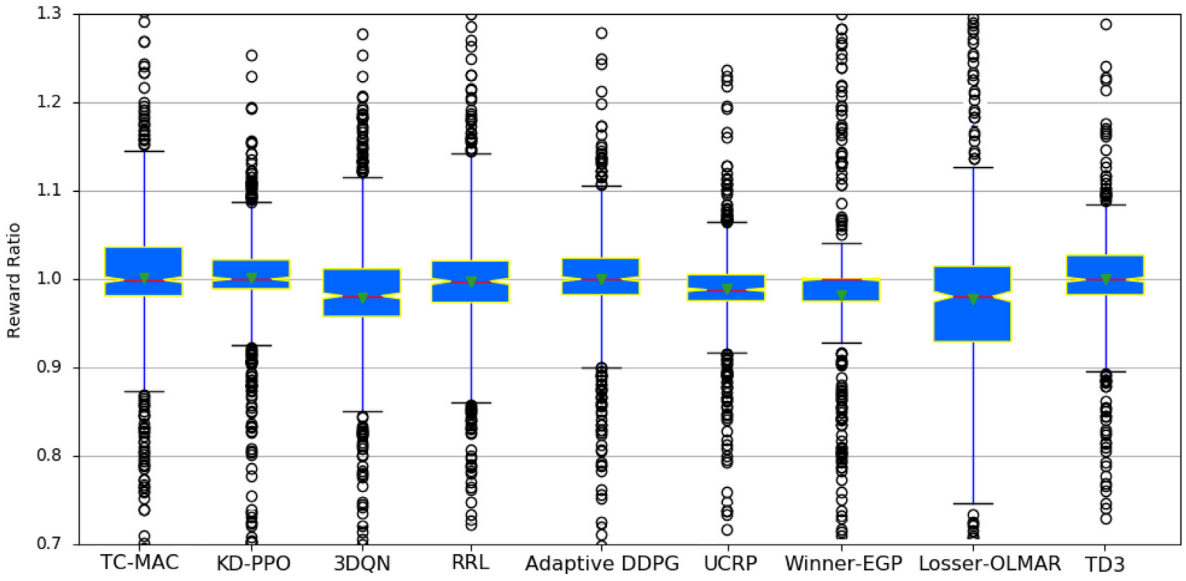


Fig. 5. Reward ratios in heterogeneous transfer.

of test episodes, and then the temporal averages are computed. It can be noted that TC-MAC algorithm outperforms (more than 35%) the state-of-the-art algorithms in terms of five metrics except for MDD, the MDD of KD-PPO is about 0.141, while that of TC-MAC is about 0.197, which suggests that KD-PPO algorithm achieves more stable compared with other algorithms. For the metrics NP and PY, TD3 achieves 2103.50 and 0.101, the corresponding values of Adaptive DDPG are respectively about 1878.65 and 0.090, while TC-MAC is much better than TD3, that of TC-MAC are about 3207.32 and 0.149, respectively. For the metrics SR and CR, KD-PPO is the best one of all compared algorithms, here SR is about 0.921 and CR is about 0.652; While TC-MAC is better than KD-PPO. Which suggests that the knowledge distillation improves the performance to some extent, but TC-MAC algorithm can work better due to the adopted context information and corresponding optimization. Although Adaptive DDPG acquires relatively high NP and PY, it suffers from more fluctuation compared with KD-PPO after achieving the stable situation. For these traditional methods, it can be noted that Loser-OLMAR is the worst one of all compared algorithms, the corresponding NP is −4975.61, which means that the final balance is US$5024.39 at the condition of initial wealth US$10000.00. Winner-EGP has achieved similar results. In addition, it can be noted that UCRP is better than 3DQN in terms of all five metrics.

To further describe the performances of these algorithms in the backtest process, both portfolio value (i.e., the accumulated wealth) and reward ratio for the initial wealth are widely used in [6,9,18,19], the corresponding curves are plotted in Figs. 4–5; In addition, the weight values for different assets is widely used in [3,4], the corresponding curve is plotted in Fig. 6.

Fig. 4 plots the curves of portfolio values for all algorithms. It can be noted that KD-PPO is better than other compared algorithms before 1/31/2019, while at the later stage, TD3 is better than other compared algorithms. Although KD-PPO is better than TC-MAC at some periods of test, but the latter is better than the former in the whole test process, which also can be seen from Table 2. Specifically, the periods from 01/01/2018 to 06/29/2018, and from 09/30/2018 to 12/30/2018, the curve of TC-MAC is below that of KD-PPO, that is to say, the portfolio value of KD-PPO is more than that of TC-MAC, but it can be seen that the standard deviation of KD-PPO is more than that of TC-MAC, which suggests that TC-MAC has stronger convergence than KD-PPO. In addition, it can be noted that curve of KD-PPO has been limited in a range after the period 06/29/2018, the limited range is about from 11034.00 to 11648.00, while the curve of TC-MAC always keeps the increasing trend, the corresponding portfolio value is about from 11650.00 to 13207.00. Which suggests that TC-MAC has the best capacity to improve portfolio value. The possible explanation is that the knowledge distillation adopted by KD-PPO can improve portfolio value to some extent, while the additional context representation and corresponding optimization adopted by TC-MAC can effectively improve portfolio value. For RRL and Adaptive DDPG, it can be noted that there exists larger gap between the corresponding curves at the early period, but they become pretty close at the following periods, the portfolio values of them are limited from 10000.00 to 11678.00 after the period 05/31/2018, which suggests that both have the similar and effective performance. While for 3DQN, portfolio value is always below the initial wealth 10000.00, which means that 3DQN does not produce effective portfolio policy compared with other DRL algorithms. Moreover, for these three traditional methods, especially Loser-OLMAR and Winner-EGP, the produced portfolio policies do not work well enough, which causes portfolio values to decrease, which means that following winner or loser cannot suit to the complex financial market. For UCRP, it can be noted that the corresponding curve is above 10000.00 at some periods, i.e., from 06/24/2018 to 10/3/2018, 03/18/2019 to 04/29/2019, and after the period 09/30/2019; While duration other periods, the curve is below 10000.00, which means that this method suffers from relatively large fluctuations, the reason may be the same weight policies induced by the uniform distribution.

Fig. 5 presents the distribution of reward ratios for all algorithms during the heterogeneous transfer. It can be noted that TC-MAC, Adaptive DDPG, KD-PPO and RRL have the same median values 1.0 and mean values 1.0, but these algorithms have different Q3 and Q1 values. Specifically, TC-MAC has Q3 value 1.048 and Q1 value 0.985, TD3 has Q3 value 1.039 and Q1 value 0.982, RRL has Q3 value 1.032 and Q1 value 0.973, KD-PPO has Q3 value 1.019 and Q1 value 0.99, Adaptive DDPG has Q3 value 1.022 and Q1 value 0.982. Based on the above numeral results, compared with other algorithms, TC-MAC has more reward ratios of over 1.0, which makes the curve of TC-MAC always increase; In addition, about IQR=Q3-Q1, TC-MAC has 0.063, RRL has 0.059, Adaptive DDPG has 0.04, and KD-PPO has 0.029, KD-PPO has the

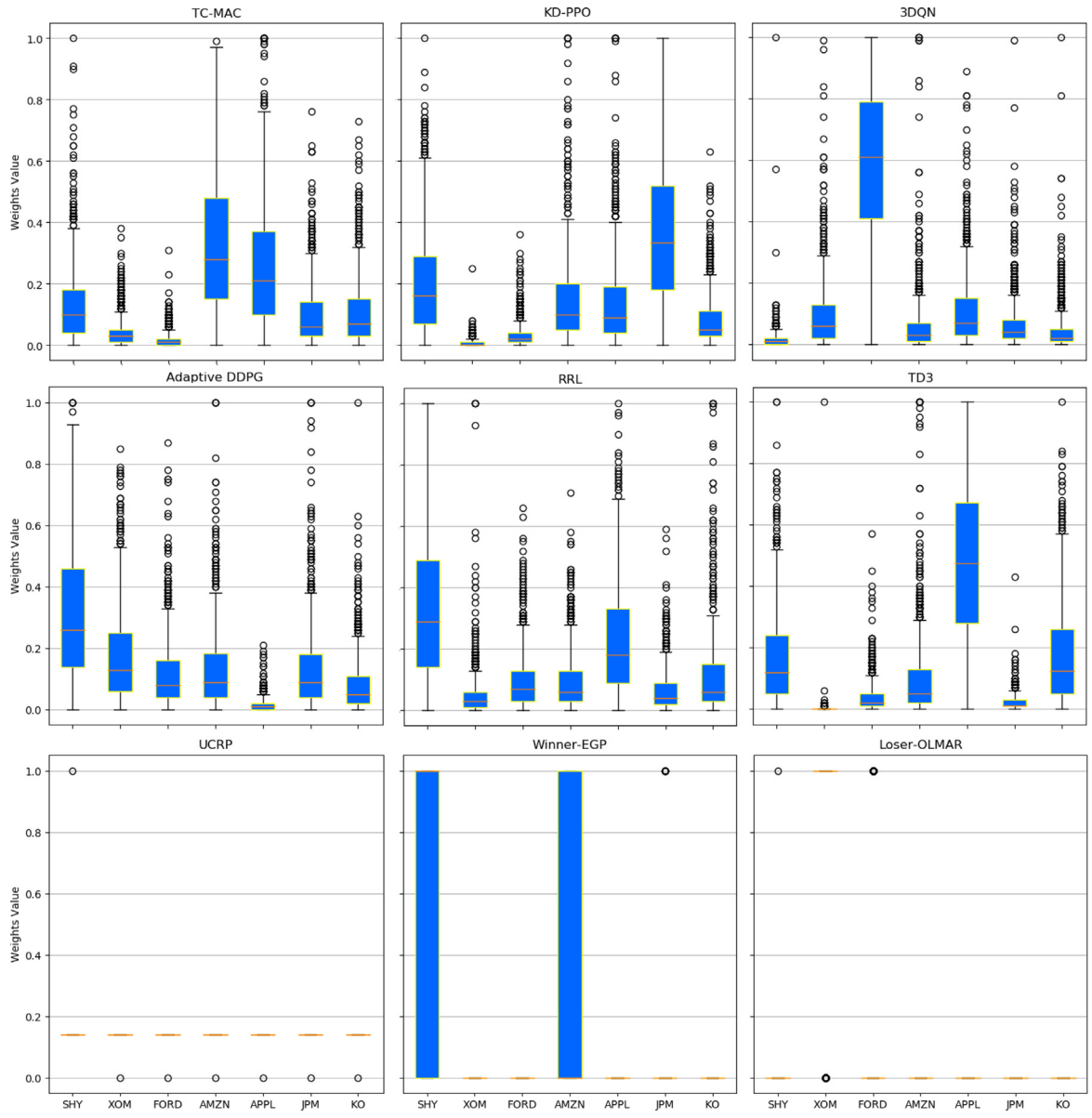


Fig. 6. Weight values for different assets in heterogeneous transfer.

smallest IQR, which makes the curve of KD-PPO keep in the limited range. For Q1, Q3 and IQR, Adaptive DDPG and RRL have the similar values, which means that they have similar performance, so their curves for portfolio values are relatively close. Although Winner-EGP has the median value 1.0, the corresponding Q3 and mean values are respectively 1.0 and 0.985, which indicates that more than 75% reward ratios is less 1.0. This leads to the portfolio values of Winner-EGP always less than the initial wealth 10000.00, which can be also seen from Fig. 3. In addition, 3DQN, UCRP and Loser-OLMAR have the median is less 1.0; 3DQN has Q3 value 1.015 and Q1 value 0.962, UCRP has Q3 value 1.005 and Q1 value 0.975, Loser-OLMAR has Q3 value 1.029 and Q1 value 0.939, it can be noted the most parts of IQRs are less than 1.0, which leads to that the corresponding portfolio values are less than the initial wealth 10000.00. While UCRP has some portfolio values larger than 10000.00 due to the limited range of IQR (i.e., 0.03). Based on the above analysis, the results of reward ratio, i.e., boxplots, means and medians, suggest that TC-MAC algorithm significantly outperforms other methods.

Fig. 6 presents the distribution of different assets' weight values for all algorithms during the heterogeneous transfer. About the trends of these different assets, the details are described as follows: the periods from 01/01/2018 to 12/31/2019, XOM and FORD have the downward trends, while Apple and Amazon have the upward trends; In addition, JPM and KO have the upward trends at the early periods from 01/01/2018 to 06/30/2019, and downward trends at the periods from 07/01/2018 to 12/31/2019. Specifically, about the weight values of Amazon and Apple, it can be noted that TD3 achieves 0.475 and 0.045, the corresponding median values of KD-PPO are respectively 0.089 (Amazon) and 0.077(Apple); TC-MAC has the median values respectively 0.295 (Amazon) and 0.213(Apple), which suggests that TC-MAC is better than both TD3 and KD-PPO about choosing the higher-yielding assets. Adaptive DDPG has the median values respectively 0.105 (Amazon) and 0.012(Apple), RRL has the median values respectively 0.069 (Amazon) and 0.067 (Apple), while both have the similar performance, the major reasons are: 1) RRL can effectively trade off the assets Amazon and Apple compared

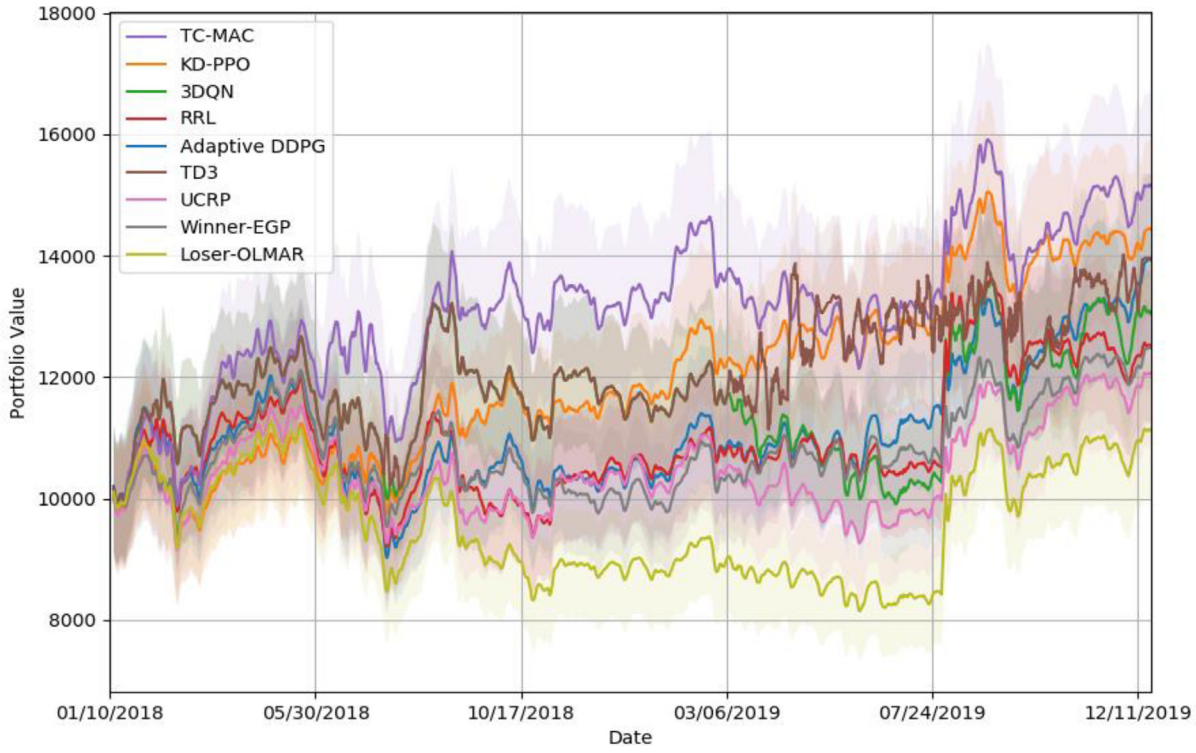


Fig. 7. Portfolio values in homogeneous transfer.

with Adaptive DDPG, 2) RRL has the lower weight value for XOM compared with Adaptive DDPG. It also can be noted that TC-MAC and KD-PPO have the low weight values for XOM and FORD due to these assets with downward trends, which helps to improve portfolio values, this is also illustrated by Fig. 4. Although 3DQN has respectively 0.021(Amazon) and 0.034 (Apple), the weight values of XOM and FORD are high, especially the latter one, which leads to its poor overall performance, the corresponding portfolio value plotted in Fig. 4 also illustrates that. About assets JPM and KO, these algorithms have the weight values less than 0.19 except for KD-PPO, which has the higher weight value for JPM. In addition, it can be noted that UCRP has the same weight value 0.143 for each asset; Winner-EGP has the median values 0.0 for each asset (e.g., Amazon and Apple) except for SHY, but IQR=Q3-Q1 for Amazon is 1.0, the same situation also happens to the asset SHY, that is because this method follows the winners; Loser-OLMAR has the median value 0.0 for each asset expect XOM, the median value of XOM is 1.0, which means this method only choice XOM due to the principle following the loser.

The heterogeneous transfer results have been analyzed in the above. Due to the similarity between the curves/boxplots in heterogeneous transfer and those in homogeneous transfer, in the next section, we will only describe the differences between them to improve readability and avoid redundancy.

4.5.2. Homogeneous transfer

Table 3 sums up five performance metrics of TC-MAC algorithm and other compared algorithms in homogeneous transfer. The results in Table 3 are acquired by the same methods calculating those in Table 2. Similar to results in heterogeneous transfer, it can be noted that TC-MAC algorithm is also better than all compared algorithms in homogeneous transfer.

Compared the results (i.e., Table 2) in heterogeneous transfer with those (i.e., Table 3) in homogeneous transfer. It can be noted that KD-PPO is the best one of all compared algorithms in terms of NP, PY and SR; While about the metrics of CR and MDD, TD3 is the best one of all compared algorithms; TC-MAC is better than both KD-PPO and TD3 in terms of five metrics in homogeneous transfer compared with these metrics except for MDD in heterogeneous transfer, which suggests that TC-MAC achieves the effective transferability compared with other algorithms in homogeneous transfer. Specifically, TC-MAC has NP values about 5168.83 (homogeneous transfer) and 3207.32 (heterogeneous transfer), PY values about 0.232 (homogeneous transfer) and 0.149 (heterogeneous transfer), SR values about 2.578 (homogeneous transfer) and 1.518 (heterogeneous transfer), and CR values about 1.059 (homogeneous transfer) and 0.756 (heterogeneous transfer). In addition, for 3DQN, Winner-EGP and Loser-OLMAR, we can note that the values of metrics (i.e., NP, PY, SR and CR) are all negative values in heterogeneous transfer, while those are all positive values in homogeneous transfer, especially 3DQN achieves the relatively better performance than all traditional methods and RRL, which suggests that 3DQN is more suit to the homogeneous transfer compared with heterogeneous transfer. Based on the above analysis, we can note that the all-metric values of TC-MAC in homogeneous transfer are larger than those in heterogeneous transfer, the similar situation also happens to other compared algorithms, thus the above explanations also suit for them. The major reason is that the homogeneous transfer adopts the same type assets used in training test (Note that the period is different), while the heterogeneous transfer uses totally different type and quantity assets. Although the former is relatively effective, the latter is more challenging but

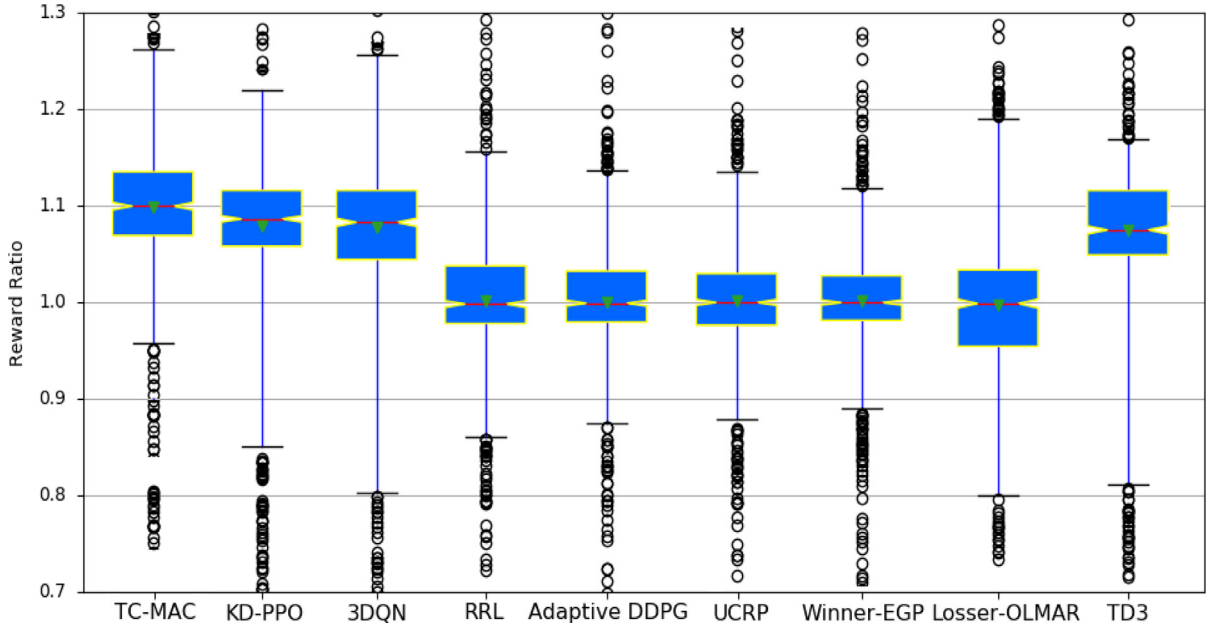


Fig. 8. Reward ratios in homogeneous transfer.

practical, which further measures these algorithms' capacity to generalize to scenarios with diverse assets.

Compared Fig. 7 with Fig. 4 for portfolio values, it can be seen that the corresponding curve for the same algorithm in homogeneous transfer is better than that in heterogeneous transfer. KD-PPO is the best one of all compared algorithm in homogeneous transfer, TD3 can achieve relatively good performance in heterogeneous transfer. About portfolio values, KD-PPO can keep increasing in homogeneous transfer, while keep stable in heterogeneous transfer, which can be illustrated from the corresponding reward ratios respectively plotted in Figs. 8 and 5; At the same time, which also suggests that the knowledge distillation technique helps to produce relatively effective portfolio policies to some extent. TC-MAC is better than KD-PPO and TD3 in both homogeneous and heterogeneous transfers, which suggests that TC-MAC produces effective portfolio policies. Similarly, 3DQN achieves better portfolio values in homogeneous transfer than that in heterogeneous transfer, the reward ratio plotted in Fig. 8 illustrates that results, which suggests that it produces effective weights of assets in the former compare with the latter, at the same time, the corresponding weight values plotted in Figs. 9 and 5 also illustrate that, that is to say, the bullish asset (i.e., Amazon and Apple, MSFT) are with relatively high weight values, by contrast, the bearish assets (i.e., XOM and FORD, 3M) are with relatively low weight values. Compared Fig. 7 with Fig. 5, Winner-EGP can keep the positive NP (i.e., portfolio values more than 10000) in homogeneous transfer, by contrast, the portfolio values are always less than 9000 in heterogeneous transfer (i.e., negative NP), which suggests that Winner-EGP cannot well suit to the scenarios with too much assets. As shown in Figs. 7 and 5, the similar situation also happens to Loser-OLMAR, which works better in homogeneous transfer than heterogeneous transfer.

Compared Fig. 8 with Fig. 5 for reward ratio, KD-PPO and 3DQN achieve the median values respectively about 1.082 and 1.073, the former is the best one of all compared algorithm, TC-MAC achieves the median value about 1.11, and other algorithms have the median values 1.0. In addition, compared Fig. 9 with Fig. 6 for weight values, these traditional methods, i.e., UCRP, Winner-EGP and Loser-OLMAR, produce the similar weight values in homogeneous and heterogeneous transfers, but these corresponding portfolio values are totally different, as shown in Figs. 7 and 4, which suggests that these traditional methods can suit to the relatively less assets (i.e., homogeneous transfer) instead of heterogeneous transfer. For these DRL algorithms, i.e., KD-PPO, TD3, Adaptive DDPG, 3DQN and RRL, the produced weight values are diverse in homogeneous and heterogeneous transfers, which leads to the different portfolio values shown in Figs. 7 and 4, and the different reward ratios shown in Figs. 8 and 4, the possible reason is that: these traditional methods often follow certain fixed principles of weight allocation, while these DRL algorithms are first trained, which can learn capability from the training dataset.

Thus, to sum up the above analysis and results, TC-MAC algorithm outperforms all compared algorithms in multiple metrics, which demonstrates that TC-MAC algorithm not only performs effective homogeneous transfer, but also achieves reliable heterogeneous transfer.

4.6. Ablation study

The proposed TC-MAC algorithm is based on both feature representation and optimization, in which optimization is for the *representation learning* and *policy learning*, at the same time, feature representations are for both *task* of PM and *context* of portfolio. Thus, ablation studies are conducted on several variants of TC-MAC algorithm to verify the effectiveness of each part.

No-Task: the task embedding Z^t_{task} defined in Eq. (19) is ignored, the designed attention-based GCN is to learn the context embedding $Z^t_{Context}$ defined in Eq. (13), which is as the final portfolio embedding.

No-Context: the context embedding $Z^t_{Context}$ is ignored, the designed attention-based Bi-GRU is to learn the task embedding Z^t_{task}, which is as the final portfolio embedding.

No-$\mathcal{L}_{\text{AC}}$: the AC loss function defined Eqs. (29) and (28) *is ignored,* only from the respective of representation learning, i.e., the final objective function is $\mathcal{L}_{\text{MI}}$ *defined in Eq.* (25)*, which* optimizes the whole algorithm.

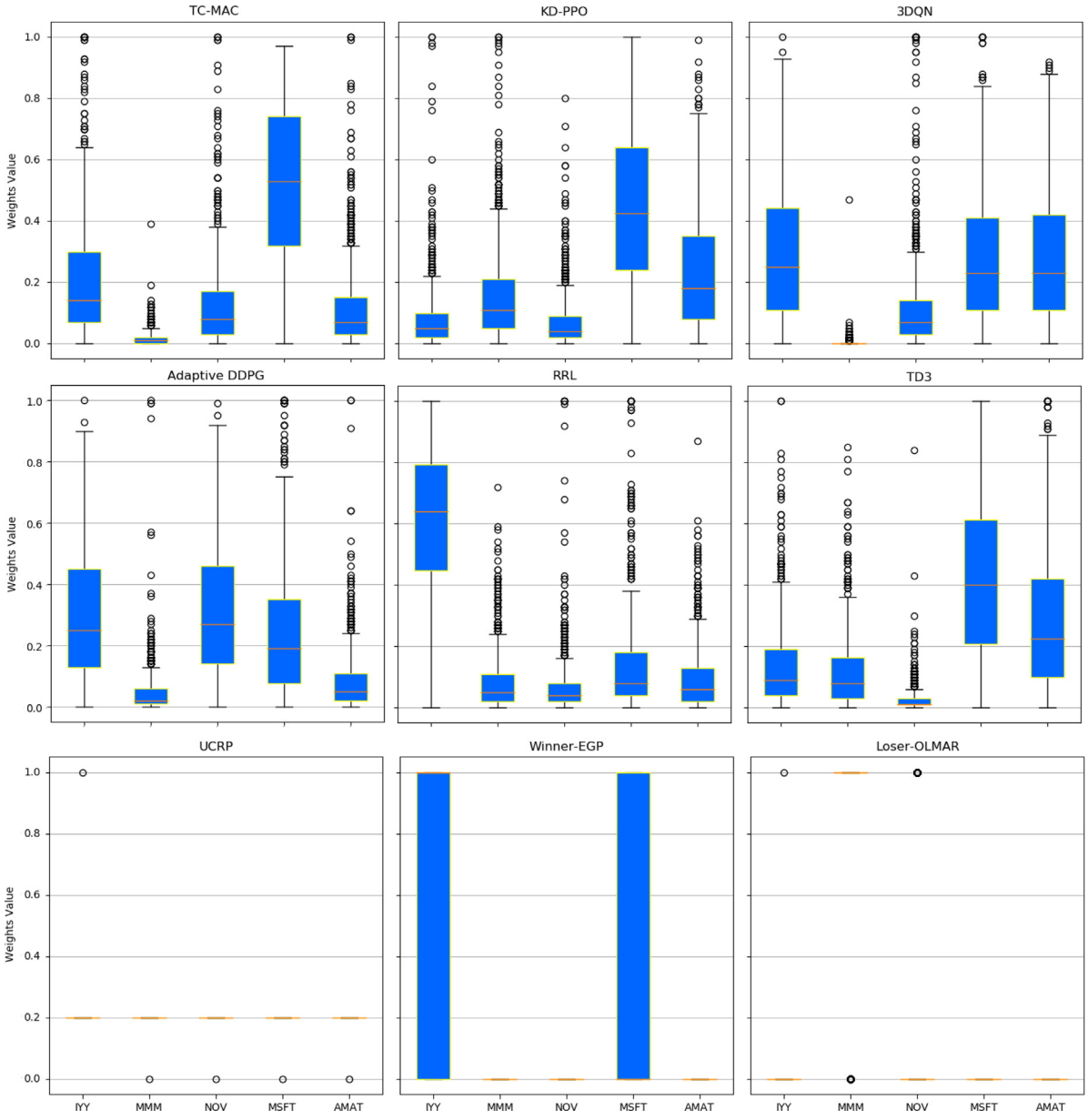


Fig. 9. Weight values for different assets in homogeneous transfer.

Table 4
Ablation results.

Metrics	Homogeneous transfer					Heterogeneous transfer				
	TC-MAC	No-$\mathcal{L}_{MI}$	No-$\mathcal{L}_{AC}$	No-Context	No-Task	TC-MAC	No-$\mathcal{L}_{MI}$	No-$\mathcal{L}_{AC}$	No-Context	No-Task
Net Profit (NP)	**5168.83**	4217.76	3148.93	3946.52	2315.16	**3207.32**	1641.52	1645.76	1376.56	1401.67
Percentage Yield (PY)	**0.232**	0.192	0.147	0.181	0.110	**0.149**	0.079	0.080	0.067	0.068
Sharpe Ratio (SR)	**2.578**	2.098	1.455	1.865	1.009	**1.518**	0.796	0.842	0.688	0.650
Calmar Ratio (CR)	**1.059**	0.756	0.488	0.678	0.315	**0.756**	0.341	0.374	0.255	0.251
Maximum Drawdown (MDD)	**0.219**	0.254	0.301	0.267	0.349	**0.197**	0.232	0.214	0.263	0.271

No-$\mathcal{L}_{\mathrm{MI}}$: the MI loss function is ignored, only from the respective of RL, the final objective function is *defined in* Eqs. (29) and (28) *without the term of* $\mathcal{L}_{\mathrm{MI}}$, *which* optimizes the whole algorithm.

Table 4 summarizes the results of five performance metrics for these variant algorithms in homogeneous transfer and heterogeneous transfer, respectively.

In homogeneous transfer, for these five metrics, the numeral results show that *No-*$\mathcal{L}_{\mathrm{MI}}$ *is the best one of these variant algorithms.* From the viewpoint of optimization, it can be seen that *No-*$\mathcal{L}_{\mathrm{MI}}$ is better than *No-*$\mathcal{L}_{\mathrm{AC}}$ in terms of all metrics, e.g., PY values for *No-*$\mathcal{L}_{\mathrm{MI}}$ and *No-*$\mathcal{L}_{\mathrm{AC}}$ are respectively about 0.192 and 0.147, which suggests that RL optimization makes larger contribution than the representation optimization via maximizing mutual information, but the latter still improves the performance of the whole algorithm to some extent; And from the view of feature representation, No-Context is better than No-Task in terms of five metrics, e.g., MDD values for No-Context and No-Task are

respectively about 0.267 and 0.349, which suggests that the task representation of asset allocation is more effective than the context representation of the whole portfolio. In addition, it can be noted that No-Task is the worse one of the four variant algorithms in terms of five metrics, which suggests that the missed part (i.e., the task representation of asset allocation) makes the largest contributions compared with other three missing parts (i.e., RL optimization, the context representation of portfolio and corresponding optimization).

In heterogeneous transfer, for these five metrics, *No-*$\mathcal{L}_{\mathrm{AC}}$ *is the best one of these variant algorithms.* From the viewpoint of optimization, the performance of *No-*$\mathcal{L}_{\mathrm{MI}}$ is pretty close that of *No-*$\mathcal{L}_{\mathrm{AC}}$, which can be demonstrated by the values of different metrics, e.g., NP values for *No-*$\mathcal{L}_{\mathrm{MI}}$ and *No-*$\mathcal{L}_{\mathrm{AC}}$ are about 1641.52 and 1645.76, respectively. In addition, it also can be noted that the same situation happens to the feature representation variant algorithms, i.e., No-Context and No-Task have the pretty close values for these different metrics. On the contrary, we can note that in homogeneous transfer, the values of different metrics for the optimizations (i.e., *No-*$\mathcal{L}_{\mathrm{MI}}$ and *No-*$\mathcal{L}_{\mathrm{AC}}$) and feature representations (i.e., No-Context and No-Task) are large different compared with these variant algorithms in heterogeneous transfer, e.g., CR values for *No-*$\mathcal{L}_{\mathrm{AC}}$ and *No-*$\mathcal{L}_{\mathrm{MI}}$ are about 0.374 and 0.341, respectively. And CR values for No-Context and No-Task are about 0.255 and 0.251, respectively. Thus, which suggests that: on the one hand, context representation of portfolio are as effective as the task representation of asset allocation, and the corresponding objective function $\mathcal{L}_{\mathrm{AC}}$ *can derive the similar optimization compared with* $\mathcal{L}_{\mathrm{MI}}$; On the other hand, the missing parts (i.e., context representation and the corresponding objective function) indeed improve the performance of the whole algorithm to some extent.

Compared the results between homogeneous transfer and heterogeneous transfer, for No-$\mathcal{L}_{\mathrm{MI}}$, NP values are respectively about 4217.76 and 1641.52, SR values are respectively about 2.098 and 0.796, and MDD values are respectively about 0.254 and 0.232. For No-Context, NP values are respectively about 3946.52 and 1376.56, SR values are respectively about 1.865 and 0.688, and MDD values are respectively about 0.267 and 0.263. It can be noted that the values for the same metric are large different, which illustrates that the homogeneous transfer can achieve more effective results compared to the heterogeneous transfer. The major reason is that the homogeneous transfer uses the same assets for both training and test, while the heterogeneous transfer uses entirely different assets for both.

Based on the above analysis, which suggests that context representation for portfolio and optimization for representation learning indeed help to improve the performance of the whole algorithm to varying degrees. The numeral results clearly corroborate the effectiveness of different parts from the whole algorithm to improve the overall performance of TC-MAC algorithm.

5. Related work

To achieve effective PM, the wealth should be allocated to the high-return and low-risk multiple assets. So different PM methods have been proposed in the past decades. The traditional methods, such as Markowitz [33] and Risk Budgeting [6], aim to maximize the expected returns and minimize risks for PM. The Markowitz-based methods employ the historical prices of different assets to guess at the resultful margins, and derive the optimal portfolio [33]. [1] designs an algorithm based on newton method for asset portfolio, which can yield higher returns than previous algorithms, but run relatively faster. The Risk-Budgeting based methods guess at the portfolio weights by allocating the risks to different assets [3]. In [4], the method is based on minimum variance and equally-weighted portfolios, in which the risk contribution from each asset is equal to maximize diversification of risks. In [5], an on-line portfolio selection method, i.e., OLMAR, is designed, which is based on a multiple-period mean reversion (i.e., MAR) via some powerful online learning techniques. In [6], a stochastic risk budgeting method is designed for optimization, which employs the portfolio and the marginal risk constraint. In general, these traditional methods suffer from the issues of flexibility in the goals setting and diverse time-sequential information encoding.

In recent years, the implementation of machine learning has been actively studied in PM. In order to encode the time -sequential data, [7] develops a portfolio construction method based on both a machine learning model for stock prediction and Mean–Variance (MV) model for portfolio selection. Another research [8] employs sentiment analysis of social media to predict the assets' prices. Anther similar research [9] employs different machine learning techniques, such as LSTM, Random Forest (RF) and Support Vector Regression (SVR), to predict the expected returns of portfolio. [10] designs a neural network-based portfolio optimization model to predict the fluctuation of market, which aim to acquire the optimal asset portfolio based on both the price-volume data and the macroeconomic data. However, these above-mentioned algorithms do not consider the interaction with time-vary financial market, which lead to that the dynamic portfolio management cannot be described effectively.

Due to action–reward mechanism [32], RL has been applied into different areas, e.g., epidemic control [44] and traffic signal control [45]. Here, dozens of RL-based methods are designed for PM, deep neural networks further improve the capability. [9] proposes a recurrent RL algorithm, which updates the allocated weights of assets using the actions of buy and sell. [19] proposes a deep RL algorithm, which employs a CNN to capture the features of cryptocurrency portfolio management. And other research [18] based on deep deterministic RL scheme, can adapt to different PM via capturing the features about the optimistic or pessimistic information from financial market. [17] proposes a deep RL algorithm based on the knowledge distillation, which aims to enhance the training reliability and improve the performances in the backtest. [12] proposes an asset portfolio RL algorithm using the stock prices and signals of market fluctuation, which is learned from the financial article data and stock price data. In [11], an algorithm based on the on-policy SARSA(λ) and off-policy Q(λ), which aims to maximize the portfolio returns and differential Sharpe ratios using the discrete action/state spaces. [14] designs an algorithm combining recurrent RL and particle swarm algorithm, which aim to maximize the investment returns via both achieved effective portfolio policy and constraint optimization. [28] designs a multi-layer and multi-ensemble stock model, which can maximize profit and generate stock signals via the pre-processing DNN and a reward-based classifier.

To summarize, deep RL has been widely put into the PM, the features of multiple assets are captured by DNNs, and the portfolio weights are learned by RL. However, some context information about portfolio is ignored. In addition, these algorithms are general optimized by RL-based loss functions, some relationships between different representations are not considered. In this paper, the above-mentioned issues have been dealt with by our proposed algorithm.

6. Conclusion

This paper proposes a TC-MAC algorithm for effective PM. Specifically, TC-MAC algorithm is developed based on: 1) *representation learning* adopts a proposed TC learning algorithm, which

not only encodes the task (i.e., the local features of different assets) of PM as task embeddings, but also encodes the global dynamic context of portfolio as context embeddings, thus makes for producing optimal portfolio embeddings; 2) *policy learning* adopts a proposed MAC framework, which calculates the relationships between local embedding of each asset and global context embeddings by maximizing mutual information, the corresponding MI loss function combines with AC loss function to collectively optimize the whole algorithm, thus makes for deriving optimal portfolio policies. Extensive experiments are conducted on real-world datasets. Results demonstrate the superior performance of TC-MAC algorithm over other state-of-the-art algorithms in terms of multiple metrics and transferability.

Despite different RL algorithms for PM have been paid much attention and research in recent years, it is still a new developing area. In the future work, how to further improve the feature representations for different assets, which is a research direction because effective representation will do good for allocating the weights of assets. In addition, other side information, such as different news about these assets, should be considered, which makes for modeling more real financial market. Furthermore, multi-agent RL has been studied in depth [46], how to apply the multi-agent RL to PM, which could be a research direction due to multiple assets in PM.

CRediT authorship contribution statement

Shantian Yang: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors do not have permission to share data.

Acknowledgment

This work is supported by the Fundamental Research Funds for the Central Universities, China (Project No. JBK23YJ26).

References

[1] A. Agarwal, E. Hazan, S. Kale, R.E. Schapire, Algorithms for portfolio management based on the newton method, in: 23th International Conference on Machine Learning, Vol. 148, 2006, pp. 9–16, http://dx.doi.org/10.1145/1143844.1143846.

[2] A. Timmermann, Clive. W.J. Granger, Efficient market hypothesis and forecasting, Int. J. Forecast. 20 (1) (2004) 15–27, http://dx.doi.org/10.1016/S0169-2070(03)00012-8.

[3] T. Roncalli, G. Weisang, Risk parity portfolios with risk factors, Quant. Finance 16 (3) (2016) 377–388, http://dx.doi.org/10.1080/14697688.2015.1046907.

[4] S. Maillard, T. Roncalli, J. Teıletche, The properties of equally weighted risk contribution portfolios, J. Portf. Manag. Summer 36 (4) (2010) 60–70, http://dx.doi.org/10.3905/jpm.2010.36.4.060.

[5] B. Li, S.C.H. Hoi, On-line portfolio selection with moving average reversion, in: 29th International Conference on Machine Learning, 2012, pp. 273–280.

[6] R. Ji, M.A. Lejeune, Risk-budgeting multi-portfolio optimization with portfolio and marginal risk constraints, Ann. Oper. Res. 262 (2) (2018) 547–578, http://dx.doi.org/10.1007/s10479-015-2044-9.

[7] W. Chen, H. Zhang, L. Mehlawat, Mean–variance portfolio optimization using machine learning-based stock price prediction, Appl. Soft Comput. 100 (1) (2021) 106943, http://dx.doi.org/10.1016/j.asoc.2020.106943.

[8] T. Hai Nguyen, K. Shirai, Julien Velcin, Sentiment analysis on social media for stock movement prediction, Expert Syst. Appl. 42 (24) (2015) 9603–9611, http://dx.doi.org/10.1016/j.eswa.2015.07.052.

[9] Y. Ma, R. Han, W. Wang, Portfolio optimization with return prediction using deep learning and machine learning, Expert Syst. Appl. 165 (2021) 113973, http://dx.doi.org/10.1016/j.eswa.2020.113973.

[10] F.D. Freitas, A.F. De Souza, Ailson R De Almeida, Prediction-based portfolio optimization model using neural networks, Neurocomputing 72 (10–12) (2009) 2155–2170, http://dx.doi.org/10.1016/j.neucom.2008.08.019.

[11] P.C. Pendharkar, P. Cusatis, Trading financial indices with reinforcement learning agents, Expert Syst. Appl. 103 (2018) 1–13, http://dx.doi.org/10.1016/j.eswa.2018.02.032.

[12] Y. Ye, H. Pei, B. Wang, P. Chen, Y. Zhu, J. Xiao, B. Li, Reinforcement-learning based portfolio management with augmented asset movement prediction states, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 34, 2020, pp. 1112–1119, http://dx.doi.org/10.1609/aaai.v34i01.5462.

[13] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780, http://dx.doi.org/10.1162/neco.1997.9.8.1735.

[14] S. Almahdi, S.Y. Yang, A constrained portfolio trading system using particle swarm algorithm and recurrent reinforcement learning, Expert Syst. Appl. 130 (2019) 145–156, http://dx.doi.org/10.1016/j.eswa.2019.04.013.

[15] V. Mnih, K. Kavukcuoglu, D. Silver, et al., Human-level control through deep reinforcement learning, Nature 518 (7540) (2015) 529–533, http://dx.doi.org/10.1038/nature14236.

[16] D. Silver, A. Huang, C.J. Maddison, A. Guez, et al., Mastering the game of Go with deep neural networks and tree search, Nature 529 (7587) (2016) 484–489, http://dx.doi.org/10.1038/nature16961.

[17] A. Tsantekidis, N. Passalis, A. Tefas, Diversity-driven knowledge distillation for financial trading using Deep Reinforcement Learning, Neural Netw. 140 (2021) 193–202, http://dx.doi.org/10.1016/j.neunet.2021.02.026.

[18] X. Li, Y. Li, Y. Zhan, X. Liu, Optimistic bull or pessimistic bear: Adaptive deep reinforcement learning for stock portfolio allocation, in: 36th International Conference on Machine Learning Workshop on AI in Finance, May 2019, 2019.

[19] G. Lucarelli, M. Borrotti, A deep Q-learning portfolio management framework for the cryptocurrency market, Neural Comput. Appl. 32 (23) (2020) 17229, http://dx.doi.org/10.1007/s00521-020-05359-8.

[20] R.S. Sutton, A. David, P. Satinder, M. Yishay, Policy gradient methods for reinforcement learning with function approximation, in: 12th Conference on Neural Information Processing Systems, 2000, pp. 1057–1063.

[21] V. Mnih, A.P. Badia, M. Mirza, A. Graves, et al., Asynchronous methods for deep reinforcement learning, in: 33th International Conference on Machine Learning, Vol. 48, 2016, pp. 1928–1937.

[22] J. Chung, C. Gulcehre, K. Cho, Y. Bengio, Empirical evaluation of gated recurrent neural networks on sequence modeling, in: 28th Neural Information Processing Systems, NIPS Workshop on Deep Learning, 2014, 2014.

[23] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, et al., Attention is all you need, in: 31th International Conference on Neural Information Processing Systems, 2017, pp. 6000–6010.

[24] N.K. Thomas, M. Welling, Semi-supervised classification with graph convolutional networks, in: 5th International Conference on Learning Representations, 2017.

[25] K. Xu, W. Hu, J. Leskovec, S. Jegelka, How powerful are graph neural networks, in: 7th International Conference on Learning Representations, 2019.

[26] S. Yang, B. Yang, An inductive heterogeneous graph attention-based multi-agent deep graph infomax algorithm for adaptive traffic signal control, Inf. Fusion 88 (2022) (2022) 249–262, http://dx.doi.org/10.1016/j.inffus.2022.08.001.

[27] L. William, Y. Rex, L. Jure, Inductive representation learning on large graphs, in: 31th International Conference on Neural Information Processing Systems, 2017, pp. 1025–1035.

[28] S. Carta, A. Corriga, A. Ferreira, A.S. Podda, D.R. Recupero, A multi-layer and multi-ensemble stock trader using deep learning and deep reinforcement learning, Appl. Intell. 51 (2) (2021) 889–905, http://dx.doi.org/10.1007/s10489-020-01839-5.

[29] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: 35th International Conference on Machine Learning, Vol. 80, 2018, pp. 1861–1870.

[30] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal Policy Optimization Algorithms, 2017, arXiv preprint arXiv:1707.06347.

[31] R. Hjelm, K. Grewal, P. Bachman, et al., Learning deep representations by mutual information estimation and maximization, in: 7th International Conference on Learning Representation, 2019.

[32] R.S. Sutton, Andrew. G. Barto, Reinforcement Learning: An Introduction, MIT Press, 2018.

[33] Markowitz H., Portfolio selection, J. Finance 7 (1) (1952) 77–91.

[34] Z. Kang, C. Peng, Q. Cheng, Kernel-driven similarity learning, Neurocomputing 267 (2017) 210–219, http://dx.doi.org/10.1016/j.neucom.2017.06.005.
[35] M.M. Bronstein, I. Kokkinos, Scale-invariant heat kernel signatures for non-rigid shape recognition, IEEE Comput. Vis. Pattern Recognit. (2010) 1704–1711, http://dx.doi.org/10.1109/CVPR.2010.5539838.
[36] M.I. Belghazi, A. Baratin, S. Rajeswar, et al., Mine: mutual information neural estimation, in: 35rd International Conference on Machine Learning, Vol. 80, 2018, pp. 531–540.
[37] P. Veliḳović, W. Fedus, W.L. Hamilton, et al., Deep graph infomax, in: 7th International Conference on Learning Representation, 2019.
[38] Z. Wang, T. Schaul, M. Hessel, H.van. Hasselt, M. Lanctot, N. de Freitas, Dueling network architectures for deep reinforcement learning, in: 33th International Conference on Machine Learning, 2016, pp. 1995–2003.
[39] J. Foerster, N. Nardelli, G. Farquhar, Stabilising experience replay for deep multi-agent reinforcement learning, in: 34th International Conference on Machine Learning, Vol. 70, 2017, pp. 1146–1155.
[40] T.P. Lillicrap, J.J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, et al., Continuous control with deep reinforcement learning, in: 32th International Conference on Machine Learning, 2015.
[41] S. Fujimoto, H. Hoof, Meger D., Addressing function approximation error in actor-critic methods, in: Proceedings of the 35th International Conference on Machine Learning, Vol. 80, PMLR, 2018, pp. 1587–1596.
[42] S. Yang, B. Yang, A semi-decentralized feudal multi-agent learned-goal algorithm for multi-intersection traffic signal control, Knowl.-Based Syst. 213 (06708) (2021) http://dx.doi.org/10.1016/j.knosys.2020.106708.
[43] S. Yang, Hierarchical graph multi-agent reinforcement learning for traffic signal control, Inform. Sci. 634 (2023) 55–72, http://dx.doi.org/10.1016/j.ins.2023.03.087.
[44] X. Du, T. Liu, S. Zhao, J. Song, H. Chen, District-coupled epidemic control via deep reinforcement learning, in: 15th International Conference on Knowledge Science, Engineering and Management, Vol. 13369, KSEM 2022, 2022, pp. 417–428, http://dx.doi.org/10.1007/978-3-031-10986-7_34.
[45] Yang. S., Yang. B., Zeng. Z., Kang. Z., Causal inference multi-agent reinforcement learning for traffic signal control, Inf. Fusion 94 (2023) 243–256, http://dx.doi.org/10.1016/j.inffus.2023.02.009.
[46] J. Hu, Sun, H. Chen, S. Huang, H. Piao, Y. Chang, L. Sun, Distributional reward estimation for effective multi-agent deep reinforcement learning, in: 36th International Conference on Neural Information Processing Systems,Vol. 35, 2022, pp. 12619–12632.